

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

B.Tech (Cybersecurity)

DATABASE SYSTEMS LAB

BCSE302L

PROJECT REVIEW 2

Database Implementation

Project Title

Smart Hostel Management System

Team Members	
Harjot Singh Bagga	24BYB0115
Tanuj Sharma	24BYB0122
Daksh Khanna	24BYB0125

Index

S. No.	Component	Page Number
1	Project Introduction and Problem Statement	3
2	Functional Requirements	3
3	Final Database Design	4 – 5
4	Table Creation	6 – 7
5	Constraints	8 – 11
6	Database Population	12 – 13
7	SQL Queries	14
8	Query Outputs	15 – 21
9	Procedures	22 – 26
10	Functions	27 – 28
11	Cursor	29 – 30
12	Triggers	31 – 33
13	CRUD Demonstrations	34
14	Database Connectivity	35 – 36
15	Real-World Transaction Demonstrations	37 – 40
16	Problems Encountered and Solutions	41 – 43
17	Conclusion	44

1. Project Introduction and Problem Statement

Problem statement. In most institutions hostel operations still run on paper registers and separate spreadsheets. Room allotment, maintenance complaints, leave applications and visitor entries are recorded in different places and are not linked to each other, so vacancy information is unreliable, complaints have no clear owner, approvals are hard to trace and reports need manual effort.

Proposed system. The Smart Hostel Management System (SHMS) stores all hostel data in one Oracle database and exposes it through a Spring Boot REST API and a React web application for three kinds of users: students, wardens/administrators and maintenance staff. The database is the core of the system: business rules are enforced by constraints, the main operations are implemented as PL/SQL procedures, and triggers keep derived data (room occupancy, resolution time, visitor QR codes) correct automatically.

This review documents the **implementation** of the database designed in Review 1: the tables and constraints that were created, the data that was loaded, the SQL queries, the procedures, functions, cursors and triggers, and how the application uses them in real transactions. All outputs in this document were produced by running the scripts against the project database (Oracle Database 23ai Free in Docker, schema `user1`).

Objectives

- Maintain a central database of hostels, rooms, students and staff.
- Allocate beds safely: never over capacity, never two active rooms for one student, never the wrong gender hostel.
- Track complaints from registration to resolution, with the responsible staff member.
- Digitise leave applications and visitor passes with approval records and QR-based entry.
- Produce occupancy, complaint, workload, leave, visitor and fee reports directly from the database.

2. Functional Requirements

The functional requirements come from the Software Engineering project document (FR-01 to FR-10) and the Review 1 requirement analysis.

ID	Requirement	Description	Main tables
FR-01	Login & registration	Students (registration no.) and staff (e-mail) log in; passwords stored as BCrypt hashes; students can self-register.	STUDENT, STAFF
FR-02	Room allocation & vacancy	Admin/warden allots a bed to a waiting student; vacancy shown per room; rooms can be vacated with history kept.	ROOM_ALLOCATION, ROOM, HOSTEL
FR-03	Complaint management	Students raise complaints; warden assigns them to matching maintenance staff; staff update status until resolved.	COMPLAINT, STAFF
FR-04	Leave management	Students apply for leave; warden approves or rejects; approval letter is generated.	LEAVE_REQUEST
FR-05	Visitor management	Students request visitor passes; warden approves; a unique QR code is used for entry and exit.	VISITOR_PASS
FR-06	Fee tracking	Students see dues and payment history; office records payments.	FEE_PAYMENT (extension)
FR-07	Announcements	Wardens post notices for one hostel or all hostels.	ANNOUNCEMENT (extension)
FR-08	Marketplace	Students list items for sale and mark them sold.	MARKETPLACE_LISTING (extension)
FR-09	Inventory	Hostel items with total and damaged quantities.	INVENTORY_ITEM (extension)

ID	Requirement	Description	Main tables
FR-10	Reports & dashboard	Occupancy, complaints, workload, leave, visitor and fee reports.	all tables
—	Lost & found, mess menu, emergency contacts	Supporting hostel-life modules.	LOST_FOUND_ITEM, MESS_MENU, STAFF

Roles

Role	What the role can do
Student	View own room and roommates, raise/track complaints, apply for leave, request visitor passes, view fees, notices, mess menu, marketplace, lost & found, emergency contacts.
Admin / Warden	Manage students, staff, hostels and rooms; allocate and vacate rooms; assign complaints (manually or automatically); decide leave and visitor requests; gate check-in; record fee payments; notices, mess menu, inventory; reports.
Maintenance	See assigned complaints, schedule visits, update status to In Progress / Resolved, upload completion photos, view inventory.

3. Final Database Design

The final design is the eight-table core schema approved in Review 1 (Figure 1). During implementation the following refinements were made; they add attributes or stricter constraints but do not change any entity or relationship:

Refinement	Reason
`STUDENT.password_hash`, `STAFF.password_hash`	Application login; only a BCrypt hash is stored.
`COMPLAINT.scheduled_date, image_path, completion_image_path`	Maintenance scheduling and photo upload (SE requirement).
`STAFF.email` NOT NULL	E-mail is the staff login username.
CHECK room type ↔ capacity; CHECK role ↔ specialization	Single = 1 bed … Quad = 4 beds; only maintenance staff have a specialization.
Function-based UNIQUE indexes on ROOM_ALLOCATION	Implements Review 1's rule "at most one Active allocation per student and per bed" so it can never be bypassed.

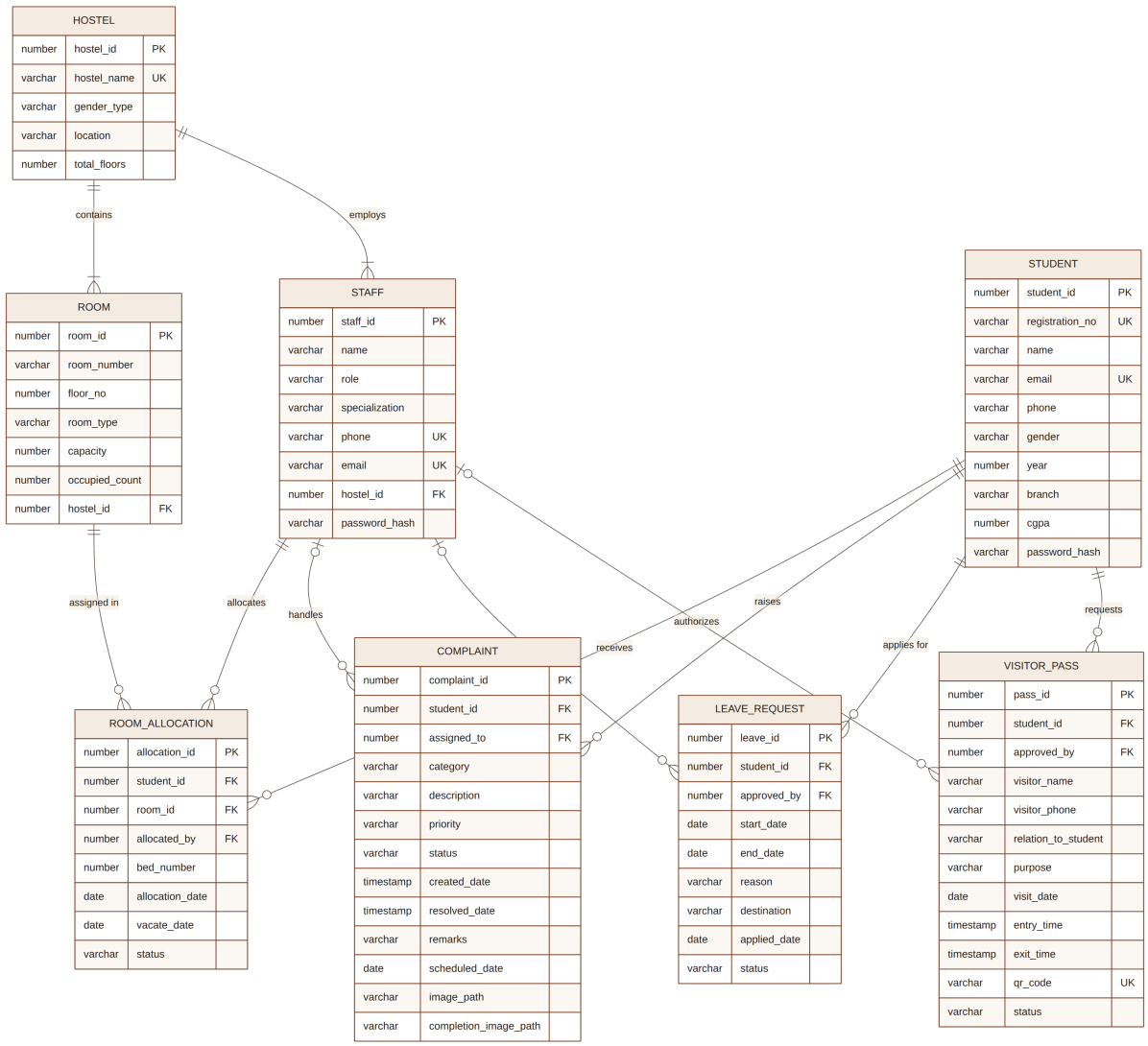


Figure 1: ER diagram of the core schema (crow's-foot notation, PK/FK/UK marked)

Relationships and cardinality

Relationship	Cardinality	Foreign key
HOSTEL contains ROOM	1 : N (total)	ROOM.hostel_id
HOSTEL employs STAFF	1 : N (total)	STAFF.hostel_id
STUDENT receives ROOM_ALLOCATION	1 : N	ROOM_ALLOCATION.student_id
ROOM assigned in ROOM_ALLOCATION	1 : N	ROOM_ALLOCATION.room_id
STAFF allocates ROOM_ALLOCATION	1 : N	ROOM_ALLOCATION.allocated_by
STUDENT raises COMPLAINT	1 : N	COMPLAINT.student_id
STAFF handles COMPLAINT	0..1 : N	COMPLAINT.assigned_to (NULL until assigned)
STUDENT applies for LEAVE_REQUEST	1 : N	LEAVE_REQUEST.student_id
STAFF approves LEAVE_REQUEST	0..1 : N	LEAVE_REQUEST.approved_by
STUDENT requests VISITOR_PASS	1 : N	VISITOR_PASS.student_id
STAFF authorizes VISITOR_PASS	0..1 : N	VISITOR_PASS.approved_by

Application extension tables

Six further tables support Software Engineering modules (fees, notices, marketplace, inventory, lost & found, mess menu). They are **not** part of the 6–8 table DBMS core; each only references a core table, so the core relations are unchanged (Figure 2).

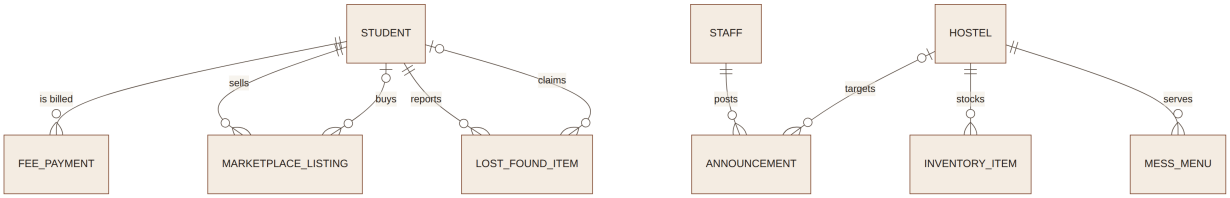


Figure 2: Extension tables and the core tables they reference

Normal form

Every table has a single-column surrogate primary key and every non-key attribute depends only on that key, so the schema is in 3NF. `ROOM.occupied_count` is the only stored derived value; it is maintained exclusively by a trigger. Fee status is deliberately *not* stored (it is derived from `amount_paid` and `amount_due`), which avoids a transitive dependency.

4. Table Creation

Tables are created by `database/schema/01_core_schema.sql`. Primary keys, NOT NULL and DEFAULT are declared with the columns; identity columns generate the keys. The complete script for the eight core tables is reproduced below; the extension tables are in `03_extension_tables.sql`.

```
-- ========================================================================
-- Smart Hostel Management System
-- 01_core_schema.sql : the 8 core tables from Review 1
-- Primary keys, NOT NULL and DEFAULT are defined here.
-- Foreign keys, UNIQUE and CHECK constraints are added in 02_core_constraints.sql
-- ========================================================================

-- Hostel blocks
CREATE TABLE hostel (
    hostel_id      NUMBER(4)     GENERATED BY DEFAULT AS IDENTITY,
    hostel_name    VARCHAR2(60)  NOT NULL,
    gender_type    VARCHAR2(6)   NOT NULL,
    location       VARCHAR2(100) NOT NULL,
    total_floors   NUMBER(2),
    CONSTRAINT pk_hostel PRIMARY KEY (hostel_id)
);

-- Hostel residents (also used for student login)
CREATE TABLE student (
    student_id       NUMBER(6)     GENERATED BY DEFAULT AS IDENTITY,
    registration_no  VARCHAR2(15)  NOT NULL,
    name             VARCHAR2(100) NOT NULL,
    email            VARCHAR2(100) NOT NULL,
    phone            VARCHAR2(15)  NOT NULL,
    gender           VARCHAR2(6)   NOT NULL,
    year             NUMBER(1),
    branch           VARCHAR2(60)  NOT NULL,
    cgpa             NUMBER(4,2),
    password_hash    VARCHAR2(100) NOT NULL,      -- refinement: BCrypt hash for login
    CONSTRAINT pk_student PRIMARY KEY (student_id)
);

-- Rooms inside a hostel
CREATE TABLE room (
    room_id         NUMBER(6)    GENERATED BY DEFAULT AS IDENTITY,
    room_number     VARCHAR2(10) NOT NULL,
    floor_no        NUMBER(2),
    room_type       VARCHAR2(10),
```

```
    capacity        NUMBER(1)    NOT NULL,
    occupied_count  NUMBER(1)    DEFAULT 0 NOT NULL,   -- maintained by trigger
    hostel_id       NUMBER(4)    NOT NULL,
    CONSTRAINT pk_room PRIMARY KEY (room_id)
);

-- Admins, wardens and maintenance staff (also used for staff login)
CREATE TABLE staff (
    staff_id        NUMBER(5)     GENERATED BY DEFAULT AS IDENTITY,
    name            VARCHAR2(100) NOT NULL,
    role            VARCHAR2(12)  NOT NULL,
    specialization  VARCHAR2(15),
    phone           VARCHAR2(15)  NOT NULL,
    email           VARCHAR2(100) NOT NULL,       -- used as the login username
    hostel_id       NUMBER(4)     NOT NULL,
    password_hash   VARCHAR2(100) NOT NULL,       -- refinement: BCrypt hash for login
    CONSTRAINT pk_staff PRIMARY KEY (staff_id)
);

-- Which bed of which room is allotted to which student (associative entity)
CREATE TABLE room_allocation (
    allocation_id    NUMBER(8)   GENERATED BY DEFAULT AS IDENTITY,
    student_id       NUMBER(6)   NOT NULL,
    room_id          NUMBER(6)   NOT NULL,
    allocated_by     NUMBER(5)   NOT NULL,
    bed_number       NUMBER(1)   NOT NULL,
    allocation_date  DATE        DEFAULT SYSDATE NOT NULL,
    vacate_date      DATE,
    status           VARCHAR2(10) DEFAULT 'Active' NOT NULL,
    CONSTRAINT pk_room_allocation PRIMARY KEY (allocation_id)
);

-- Maintenance complaints
CREATE TABLE complaint (
    complaint_id           NUMBER(8)      GENERATED BY DEFAULT AS IDENTITY,
    student_id             NUMBER(6)      NOT NULL,
    assigned_to            NUMBER(5),                       -- NULL until assigned
    category               VARCHAR2(15)   NOT NULL,
    description            VARCHAR2(1000) NOT NULL,
    priority               VARCHAR2(6)    DEFAULT 'Medium' NOT NULL,
    status                 VARCHAR2(12)   DEFAULT 'Open' NOT NULL,
    created_date           TIMESTAMP      DEFAULT SYSTIMESTAMP NOT NULL,
    resolved_date          TIMESTAMP,
    remarks                VARCHAR2(500),
    scheduled_date         DATE,                            -- refinement: maintenance schedule
    image_path             VARCHAR2(255),                   -- refinement: photo by student
    completion_image_path  VARCHAR2(255),                   -- refinement: proof of work
    CONSTRAINT pk_complaint PRIMARY KEY (complaint_id)
);

-- Leave applications
CREATE TABLE leave_request (
    leave_id      NUMBER(8)     GENERATED BY DEFAULT AS IDENTITY,
    student_id    NUMBER(6)     NOT NULL,
    approved_by   NUMBER(5),                                -- NULL while pending
    start_date    DATE          NOT NULL,
    end_date      DATE          NOT NULL,
    reason        VARCHAR2(300) NOT NULL,
    destination   VARCHAR2(100),
    applied_date  DATE          DEFAULT SYSDATE NOT NULL,
    status        VARCHAR2(10)  DEFAULT 'Pending' NOT NULL,
    CONSTRAINT pk_leave_request PRIMARY KEY (leave_id)
);

-- Visitor entry passes
CREATE TABLE visitor_pass (
    pass_id              NUMBER(8)     GENERATED BY DEFAULT AS IDENTITY,
    student_id           NUMBER(6)     NOT NULL,
    approved_by          NUMBER(5),                         -- NULL while pending
    visitor_name         VARCHAR2(100) NOT NULL,
    visitor_phone        VARCHAR2(15)  NOT NULL,
    relation_to_student  VARCHAR2(30),
    purpose              VARCHAR2(200),
    visit_date           DATE          NOT NULL,
    entry_time           TIMESTAMP,
    exit_time            TIMESTAMP,
    qr_code              VARCHAR2(40),                      -- generated by trigger on approval
    status               VARCHAR2(10)  DEFAULT 'Pending' NOT NULL,
```

```
    CONSTRAINT pk_visitor_pass PRIMARY KEY (pass_id)
);
```

Execution

All scripts are run in order by `database/run-all.sh` (schema → constraints → extension tables → functions → procedures → cursors → triggers → data). Every script ran without errors; the status of all PL/SQL objects is listed at the end of the Triggers section.

5. Constraints

Foreign keys, UNIQUE and CHECK constraints are added by `02_core_constraints.sql` with named constraints, so error messages identify the broken rule. The counts below were read from Oracle's data dictionary (`USER_CONSTRAINTS`, `USER_TAB_COLUMNS`) after creation.

Table	PK	FK	UNIQUE	CHECK	NOT NULL	DEFAULT
HOSTEL	1	0	1	2	4	1
ROOM	1	1	1	5	5	2
STUDENT	1	0	2	3	8	1
STAFF	1	1	2	3	7	1
ROOM_ALLOCATION	1	3	0	3	7	3
COMPLAINT	1	2	0	4	7	4
LEAVE_REQUEST	1	2	0	2	7	3
VISITOR_PASS	1	2	1	2	6	2

Table 1: Constraints per core table (from the data dictionary)

Two business rules from Review 1 are implemented as **function-based unique indexes**: `UX_ALLOC_ACTIVE_STUDENT`, `UX_ALLOC_ACTIVE_BED`. Only rows with status 'Active' produce a key, so a student can have many old (Vacated) allocations but only one Active one, and a bed can be held by only one Active allocation.

```
CREATE UNIQUE INDEX ux_alloc_active_student ON room_allocation (
    CASE WHEN status = 'Active' THEN student_id END);
CREATE UNIQUE INDEX ux_alloc_active_bed ON room_allocation (
    CASE WHEN status = 'Active' THEN room_id END,
    CASE WHEN status = 'Active' THEN bed_number END);
```

CHECK constraints

Constraint	Table	Condition
CK_COMPLAINT_CATEGORY	COMPLAINT	`category IN ('Electrical', 'Plumbing', 'Carpentry', 'Housekeeping', 'Network', 'Other')`
CK_COMPLAINT_PRIORITY	COMPLAINT	`priority IN ('Low', 'Medium', 'High')`
CK_COMPLAINT_RESOLVED	COMPLAINT	`resolved_date >= created_date`
CK_COMPLAINT_STATUS	COMPLAINT	`status IN ('Open', 'Assigned', 'In Progress', 'Resolved')`
CK_HOSTEL_FLOORS	HOSTEL	`total_floors > 0`
CK_HOSTEL_GENDER	HOSTEL	`gender_type IN ('Male', 'Female')`
CK_LEAVE_DATES	LEAVE_REQUEST	`end_date >= start_date`
CK_LEAVE_STATUS	LEAVE_REQUEST	`status IN ('Pending', 'Approved', 'Rejected')`
CK_ROOM_CAPACITY	ROOM	`capacity > 0`
CK_ROOM_FLOOR	ROOM	`floor_no >= 0`
CK_ROOM_OCCUPIED	ROOM	`occupied_count BETWEEN 0 AND capacity`
CK_ROOM_TYPE	ROOM	`room_type IN ('Single', 'Double', 'Triple', 'Quad')`
CK_ROOM_TYPE_CAPACITY	ROOM	`(room_type = 'Single' AND capacity = 1) OR (room_type = 'Double' AND capacity = 2) OR (room_type = 'Triple' AND capacity = 3) OR (room_type = 'Quad' AND capacity = 4)`
CK_ALLOC_BED	ROOM_ALLOCATION	`bed_number > 0`
CK_ALLOC_DATES	ROOM_ALLOCATION	`vacate_date >= allocation_date`
CK_ALLOC_STATUS	ROOM_ALLOCATION	`status IN ('Active', 'Vacated', 'Cancelled')`
CK_STAFF_ROLE	STAFF	`role IN ('Admin', 'Warden', 'Maintenance')`
CK_STAFF_ROLE_SPEC	STAFF	`(role = 'Maintenance' AND specialization IS NOT NULL) OR (role <> 'Maintenance' AND specialization IS NULL)`
CK_STAFF_SPEC	STAFF	`specialization IN ('Electrical', 'Plumbing', 'Carpentry', 'Housekeeping', 'Network')`
CK_STUDENT_CGPA	STUDENT	`cgpa BETWEEN 0 AND 10`
CK_STUDENT_GENDER	STUDENT	`gender IN ('Male', 'Female')`
CK_STUDENT_YEAR	STUDENT	`year BETWEEN 1 AND 4`
CK_VISITOR_STATUS	VISITOR_PASS	`status IN ('Pending', 'Approved', 'Rejected', 'Completed')`
CK_VISITOR_TIMES	VISITOR_PASS	`exit_time > entry_time`

Foreign keys

Constraint	Table.column	References
FK_COMPLAINT_STUDENT	COMPLAINT.STUDENT_ID	STUDENT
FK_COMPLAINT_STAFF	COMPLAINT.ASSIGNED_TO	STAFF
FK_LEAVE_STAFF	LEAVE_REQUEST.APPROVED_BY	STAFF
FK_LEAVE_STUDENT	LEAVE_REQUEST.STUDENT_ID	STUDENT
FK_ROOM_HOSTEL	ROOM.HOSTEL_ID	HOSTEL
FK_ALLOC_STAFF	ROOM_ALLOCATION.ALLOCATED_BY	STAFF
FK_ALLOC_ROOM	ROOM_ALLOCATION.ROOM_ID	ROOM
FK_ALLOC_STUDENT	ROOM_ALLOCATION.STUDENT_ID	STUDENT
FK_STAFF_HOSTEL	STAFF.HOSTEL_ID	HOSTEL
FK_VISITOR_STAFF	VISITOR_PASS.APPROVED_BY	STAFF
FK_VISITOR_STUDENT	VISITOR_PASS.STUDENT_ID	STUDENT

Constraint enforcement (actual output)

Each statement below violates one constraint; Oracle rejected all of them (block T8 of `10_demo_transactions.sql`):

```
===== T8. CONSTRAINT ENFORCEMENT =====
CHECK  cgpa = 11              -> ORA-02290: check constraint (USER1.CK_STUDENT_CGPA) violated
CHECK  room type vs capacity          -> ORA-02290: check constraint (USER1.CK_ROOM_TYPE_CAPACITY) violated
FK     room in hostel 99          -> ORA-02291: integrity constraint (USER1.FK_ROOM_HOSTEL) violated - parent
  key not found
UNIQUE duplicate staff e-mail          -> ORA-00001: unique constraint (USER1.UQ_STAFF_EMAIL) violated on table
  USER1.STAFF columns (EMAIL)
NOT NULL student name              -> ORA-01407: cannot update ("USER1"."STUDENT"."NAME") to NULL
CHECK  leave ends before start          -> ORA-02290: check constraint (USER1.CK_LEAVE_DATES) violated
UNIQUE 2nd active allocation          -> ORA-00001: unique constraint (USER1.UX_ALLOC_ACTIVE_STUDENT) violated
  on table USER1.ROOM_ALLOCATION columns (SYS_NC00009$)
FK     delete room that has allocations   -> ORA-02292: integrity constraint (USER1.FK_ALLOC_ROOM) violated -
  child record found
```

Constraint script (excerpt)

```
-- ======================================================================
-- 02_core_constraints.sql : FOREIGN KEY, UNIQUE and CHECK constraints
-- for the 8 core tables, plus supporting indexes.
-- ======================================================================

-- ---------- HOSTEL ----------
ALTER TABLE hostel ADD CONSTRAINT uq_hostel_name   UNIQUE (hostel_name);
ALTER TABLE hostel ADD CONSTRAINT ck_hostel_gender CHECK (gender_type IN ('Male', 'Female'));
ALTER TABLE hostel ADD CONSTRAINT ck_hostel_floors CHECK (total_floors > 0);

-- ---------- STUDENT ----------
ALTER TABLE student ADD CONSTRAINT uq_student_regno  UNIQUE (registration_no);
ALTER TABLE student ADD CONSTRAINT uq_student_email  UNIQUE (email);
ALTER TABLE student ADD CONSTRAINT ck_student_gender CHECK (gender IN ('Male', 'Female'));
ALTER TABLE student ADD CONSTRAINT ck_student_year   CHECK (year BETWEEN 1 AND 4);
ALTER TABLE student ADD CONSTRAINT ck_student_cgpa   CHECK (cgpa BETWEEN 0 AND 10);

-- ---------- ROOM ----------
ALTER TABLE room ADD CONSTRAINT fk_room_hostel FOREIGN KEY (hostel_id) REFERENCES hostel (hostel_id);
ALTER TABLE room ADD CONSTRAINT uq_room_number UNIQUE (hostel_id, room_number);
ALTER TABLE room ADD CONSTRAINT ck_room_floor  CHECK (floor_no >= 0);
ALTER TABLE room ADD CONSTRAINT ck_room_type   CHECK (room_type IN ('Single', 'Double', 'Triple', 'Quad'));
ALTER TABLE room ADD CONSTRAINT ck_room_capacity CHECK (capacity > 0);
ALTER TABLE room ADD CONSTRAINT ck_room_occupied CHECK (occupied_count BETWEEN 0 AND capacity);
-- capacity must agree with the room type
ALTER TABLE room ADD CONSTRAINT ck_room_type_capacity CHECK (
       (room_type = 'Single' AND capacity = 1)
    OR (room_type = 'Double' AND capacity = 2)
    OR (room_type = 'Triple' AND capacity = 3)
    OR (room_type = 'Quad'   AND capacity = 4));

-- ---------- STAFF ----------
ALTER TABLE staff ADD CONSTRAINT fk_staff_hostel FOREIGN KEY (hostel_id) REFERENCES hostel (hostel_id);
ALTER TABLE staff ADD CONSTRAINT uq_staff_phone  UNIQUE (phone);
ALTER TABLE staff ADD CONSTRAINT uq_staff_email  UNIQUE (email);
ALTER TABLE staff ADD CONSTRAINT ck_staff_role   CHECK (role IN ('Admin', 'Warden', 'Maintenance'));
ALTER TABLE staff ADD CONSTRAINT ck_staff_spec   CHECK (specialization IN
    ('Electrical', 'Plumbing', 'Carpentry', 'Housekeeping', 'Network'));
-- maintenance staff must have a specialization; admins/wardens must not
ALTER TABLE staff ADD CONSTRAINT ck_staff_role_spec CHECK (
       (role = 'Maintenance' AND specialization IS NOT NULL)
    OR (role <> 'Maintenance' AND specialization IS NULL));

-- ---------- ROOM_ALLOCATION ----------
ALTER TABLE room_allocation ADD CONSTRAINT fk_alloc_student FOREIGN KEY (student_id)   REFERENCES student (
  student_id);
ALTER TABLE room_allocation ADD CONSTRAINT fk_alloc_room    FOREIGN KEY (room_id)      REFERENCES room (
  room_id);
ALTER TABLE room_allocation ADD CONSTRAINT fk_alloc_staff   FOREIGN KEY (allocated_by) REFERENCES staff (
  staff_id);
ALTER TABLE room_allocation ADD CONSTRAINT ck_alloc_bed     CHECK (bed_number > 0);
ALTER TABLE room_allocation ADD CONSTRAINT ck_alloc_dates   CHECK (vacate_date >= allocation_date);
ALTER TABLE room_allocation ADD CONSTRAINT ck_alloc_status  CHECK (status IN ('Active', 'Vacated',
  'Cancelled'));

-- At most ONE active allocation per student, and ONE active allocation per (room, bed).
-- Function-based unique indexes: rows that are not 'Active' produce NULL keys and are ignored,
-- so the allocation history (Vacated / Cancelled rows) is still allowed.
CREATE UNIQUE INDEX ux_alloc_active_student ON room_allocation (
    CASE WHEN status = 'Active' THEN student_id END);
CREATE UNIQUE INDEX ux_alloc_active_bed ON room_allocation (
    CASE WHEN status = 'Active' THEN room_id END,
    CASE WHEN status = 'Active' THEN bed_number END);

... (34 more lines in the script file)
```

6. Database Population

Dataset type: SYNTHETIC. No public dataset of hostel operations exists — room, complaint, leave and visitor records are private student information — so a realistic synthetic dataset was generated by `database/seed/generate_seed.py` (fixed random seed, so the same data is produced every time). Names come from generic Indian first/last-name lists, registration numbers follow the `YYBBBnnnn` pattern, e-mails use the fictional domain `shms.local`, and no real person's data is used.

The generator models realistic activity: 6 hostels (4 male, 2 female) with mixed room types, about 72% bed occupancy with 25 students still waiting, previous-semester allocation history, complaints whose resolution time depends on the category, leave and visitor requests spread over the semester, and fee records for two semesters. `ROOM.occupied_count` is not inserted at all — the occupancy trigger calculated it while the allocation rows were inserted, and a check query confirmed 0 mismatching rooms.

Table	Rows	Type
HOSTEL	6	core
ROOM	180	core
STUDENT	360	core
STAFF	50	core
ROOM_ALLOCATION	418	core
COMPLAINT	460	core
LEAVE_REQUEST	270	core
VISITOR_PASS	270	core
FEE_PAYMENT	1232	extension
ANNOUNCEMENT	24	extension
MARKETPLACE_LISTING	42	extension
INVENTORY_ITEM	66	extension
LOST_FOUND_ITEM	32	extension
MESS_MENU	168	extension

Table 2: Row counts read from the database (08 Oct 2026, 00:23). Every table has well over 10 records.

Sample rows (first 5 rows of each core table)

HOSTEL

hostel id	hostel name	gender type	location	total floors
1	Aravali Block	Male	North Campus, near Main Gate	5
2	Nilgiri Block	Male	North Campus, behind Library	5
3	Shivalik Block	Male	East Campus, near Sports Complex	5
4	Vindhya Block	Male	East Campus, near Food Court	5
5	Ganga Block	Female	South Campus, near Health Centre	5

ROOM

room id	room number	floor no	room type	capacity	occupied count	hostel id
1	AR-101	1	Double	2	2	1
2	AR-102	1	Triple	3	3	1
3	AR-103	1	Triple	3	3	1
4	AR-104	1	Quad	4	4	1
5	AR-105	1	Double	2	2	1

STUDENT

student id	registration no	name	gender	year	branch	cgpa	password hash
1	24BYB1001	Aarav Mehta	Male	3	CSE (Cyber Security)	8.29	$2b$10$wF/tr...
2	23BCE1008	Atharv Agarwal	Male	4	Computer Science and Engineering	8.21	$2b$10$wF/tr...
3	25BEC1008	Deepak Pandey	Male	2	Electronics and Communication	7.4	$2b$10$wF/tr...
4	23BYB1002	Vikram Rathore	Male	4	CSE (Cyber Security)	7.77	$2b$10$wF/tr...
5	26BIT1007	Aditya Chauhan	Male	1	Information Technology	7.9	$2b$10$wF/tr...

STAFF

staff id	name	role	specialization	phone	email	hostel id
1	Rajesh Kulkarni	Admin		9370283822	admin@shms.local	1
2	Suresh Iyer	Warden		7646861642	warden@shms.local	1
3	Prakash Naidu	Maintenance	Electrical	8247764725	maint@shms.local	1
4	Meera Krishnan	Admin		9718841330	meera.krishnan4@shms.local	1
5	Karan Mishra	Warden		8682052589	karan.mishra5@shms.local	1

ROOM_ALLOCATION

allocation id	student id	room id	allocated by	bed number	allocation date	vacate date	status
1	1	106	11	1	15-07-2025	10-05-2026	Vacated
2	4	59	19	1	10-07-2025	14-05-2026	Vacated
3	7	2	19	3	20-07-2025	14-05-2026	Vacated
4	11	26	19	2	10-07-2025	18-05-2026	Vacated
5	12	101	28	1	12-07-2025	19-05-2026	Vacated

COMPLAINT

complaint id	student id	assigned to	category	priority	status	created date	resolved date	scheduled date
1	1	7	Carpentry	High	Resolved	28-08-2026 11:54	29-08-2026 20:54	
2	1	7	Carpentry	Medium	Resolved	12-09-2026 09:47	15-09-2026 22:47	
3	1	10	Plumbing	Medium	In Progress	01-10-2026 08:24		03-10-2026
4	1	10	Plumbing	Medium	Assigned	04-10-2026 10:52		07-10-2026
5	1		Electrical	Medium	Open	06-10-2026 11:19		

LEAVE_REQUEST

leave id	student id	approved by	start date	end date	reason	destination	status
1	1	2	30-08-2026	02-09-2026	Medical check-up at home town	Indore	Approved
2	1	2	29-08-2026	01-09-2026	Medical check-up at home town	Mumbai	Rejected
3	1		12-10-2026	15-10-2026	Going home for Diwali festival	New Delhi	Pending
4	1		13-10-2026	16-10-2026	Sister's wedding	Mumbai	Pending
5	205	28	18-08-2026	24-08-2026	Internship interview	Bengaluru	Approved

VISITOR_PASS

pass id	student id	approved by	visitor name	relation to student	visit date	qr code	status
1	1	5	Tanmay Mehta	Father	28-09-2026	SHMS-VP-1-5D459ED1	Completed
2	1	2	Siddharth Reddy	Friend	09-10-2026	SHMS-VP-2-5D459ED1	Approved
3	1		Ishita Krishnan	Aunt	12-10-2026		Pending
4	1	5	Manav Yadav	Uncle	04-10-2026		Rejected
5	307	36	Nikhil Joshi	Cousin	23-07-2026	SHMS-VP-5-5D459ED1	Completed

Some wide tables are shown with a subset of their columns. Password hashes are shortened in this listing. All seeded application accounts use the password `Hostel@123`*, stored only as a BCrypt hash.*

7. SQL Queries

`database/queries/09_queries.sql` contains 21 queries. Q1–Q18 retrieve information and cover filtering, sorting, multi-table joins, outer joins, aggregation, GROUP BY, HAVING, subqueries (scalar, IN, NOT EXISTS, nested) and calls to stored functions. Q19–Q21 demonstrate INSERT, UPDATE and DELETE. Several of them are used by the application's Reports page.

No.	Query (SQL features)	Business purpose
Q1	Students currently living in Aravali Block (4-table join, sorted by room)	Warden's resident list for one hostel, room by room.
Q2	Rooms with free beds in each hostel (filter + function call)	Rooms that can still take students (used before allocation).
Q3	Occupancy of every hostel (GROUP BY + aggregate + function)	Hostel occupancy report for the administration.
Q4	Complaints by category and status (GROUP BY on two columns)	Which kinds of problems occur most and how many are still pending.
Q5	Categories with average resolution time above 2 days (GROUP BY + HAVING)	Categories that take too long to fix (service-level monitoring).
Q6	Unresolved complaints older than 3 days with student, room and assigned staff (5-table join, outer join)	Overdue complaints, with who is handling them, for follow-up.
Q7	Maintenance staff workload (LEFT JOIN + GROUP BY + function)	Workload of each maintenance worker, for fair assignment.
Q8	Pending leave requests waiting for a warden (join + filter + sort)	Warden's queue of leave requests to decide.
Q9	Leave approval rate of each warden (GROUP BY + HAVING)	Approval behaviour of each warden (audit).
Q10	Visitors expected in the next 7 days with approved passes	Gate list of visitors expected this week.
Q11	Visitor passes per month and status (date grouping)	Monthly visitor statistics.
Q12	Students waiting for a room (NOT EXISTS subquery)	Students still waiting for a room.
Q13	Students who raised more complaints than the average student (nested subquery in HAVING)	Students raising unusually many complaints (room may need inspection).
Q14	Rooms that are completely full in the hostel with the highest occupancy (subquery in WHERE)	Fully occupied rooms in the busiest hostel.
Q15	Room-type utilisation across all hostels (GROUP BY + ratio)	Which room types are most in demand.
Q16	Students with outstanding fees for the current semester (extension table, join + aggregate)	Fee defaulters for the current semester.
Q17	Hostel staff directory / emergency contacts (wardens and maintenance per hostel)	Hostel staff directory / emergency contacts.
Q18	Branch-wise residents and average CGPA of current residents (IN subquery + GROUP BY)	Branch-wise residents and academic profile.
Q19	INSERT a new student registration	New student registration (INSERT).
Q20	UPDATE: warden reschedules all Assigned complaints of Aravali Block to tomorrow	Rescheduling assigned maintenance jobs (UPDATE with subquery).
Q21	DELETE: remove withdrawn marketplace listings older than 30 days	Clearing old withdrawn marketplace listings (DELETE).

8. Query Outputs

Each query was executed with python-oracledb against the seeded database on 08 Oct 2026, 00:23. At most 12 rows of each result are shown. The DML statements (Q19–Q21) were rolled back afterwards.

Q1. Students currently living in Aravali Block (4-table join, sorted by room)

```
SELECT s.registration_no, s.name, r.room_number, a.bed_number, a.allocation_date
  FROM student s
  JOIN room_allocation a ON a.student_id = s.student_id AND a.status = 'Active'
  JOIN room r            ON r.room_id = a.room_id
  JOIN hostel h          ON h.hostel_id = r.hostel_id
 WHERE h.hostel_name = 'Aravali Block'
 ORDER BY r.room_number, a.bed_number
 FETCH FIRST 12 ROWS ONLY;
```

registration no	name	room number	bed number	allocation date
24BYB1001	Aarav Mehta	AR-101	1	16-07-2026
25BYB1006	Siddharth Bose	AR-101	2	24-07-2026
25BEC1022	Akash Reddy	AR-102	1	30-07-2026
26BAI1031	Mihir Menon	AR-102	2	19-07-2026
26BEC1007	Dev Yadav	AR-102	3	15-07-2026
25BCE1015	Ayaan Sethi	AR-103	1	18-07-2026
24BYB1018	Aarav Mishra	AR-103	2	19-07-2026
25BIT1067	Deepak Gupta	AR-103	3	17-07-2026
25BYB1014	Rohan Pandey	AR-104	1	16-07-2026
26BIT1051	Kabir Nair	AR-104	2	21-07-2026
25BIT1065	Aditya Verma	AR-104	3	30-07-2026
25BAI1058	Atharv Shah	AR-104	4	31-07-2026

(first 12 rows shown)

Q2. Rooms with free beds in each hostel (filter + function call)

```
SELECT h.hostel_name, r.room_number, r.room_type, r.capacity, r.occupied_count,
       fn_available_beds(r.room_id) AS free_beds
  FROM room r JOIN hostel h ON h.hostel_id = r.hostel_id
 WHERE r.occupied_count < r.capacity
 ORDER BY free_beds DESC, h.hostel_name, r.room_number
 FETCH FIRST 12 ROWS ONLY;
```

hostel name	room number	room type	capacity	occupied count	free beds
Ganga Block	GA-106	Quad	4	0	4
Ganga Block	GA-301	Quad	4	0	4
Kaveri Block	KA-104	Quad	4	0	4
Aravali Block	AR-501	Quad	4	1	3
Aravali Block	AR-504	Quad	4	1	3
Aravali Block	AR-505	Triple	3	0	3
Ganga Block	GA-405	Triple	3	0	3
Ganga Block	GA-406	Quad	4	1	3
Kaveri Block	KA-503	Quad	4	1	3
Nilgiri Block	NI-105	Quad	4	1	3
Nilgiri Block	NI-201	Triple	3	0	3
Shivalik Block	SH-206	Quad	4	1	3

(first 12 rows shown)

Q3. Occupancy of every hostel (GROUP BY + aggregate + function)

```
SELECT h.hostel_name, h.gender_type, COUNT(r.room_id) AS rooms,
       SUM(r.capacity) AS beds, SUM(r.occupied_count) AS occupied,
       SUM(r.capacity) - SUM(r.occupied_count) AS vacant,
       fn_hostel_occupancy_pct(h.hostel_id) AS occupancy_pct
  FROM hostel h JOIN room r ON r.hostel_id = h.hostel_id
 GROUP BY h.hostel_id, h.hostel_name, h.gender_type
 ORDER BY occupancy_pct DESC;
```

hostel name	gender type	rooms	beds	occupied	vacant	occupancy pct
Nilgiri Block	Male	30	73	58	15	79.45
Vindhya Block	Male	30	76	58	18	76.32
Kaveri Block	Female	30	70	52	18	74.29
Shivalik Block	Male	30	81	55	26	67.9
Aravali Block	Male	30	84	57	27	67.86
Ganga Block	Female	30	84	55	29	65.48

Q4. Complaints by category and status (GROUP BY on two columns)

```
SELECT category,
       COUNT(*) AS total,
       SUM(CASE WHEN status = 'Resolved' THEN 1 ELSE 0 END) AS resolved,
       SUM(CASE WHEN status <> 'Resolved' THEN 1 ELSE 0 END) AS pending
  FROM complaint
 GROUP BY category
 ORDER BY total DESC;
```

category	total	resolved	pending
Electrical	129	117	12
Housekeeping	85	78	7
Plumbing	83	67	16
Network	64	55	9
Carpentry	61	47	14
Other	38	32	6

Q5. Categories with average resolution time above 2 days (GROUP BY + HAVING)

```
SELECT category, COUNT(*) AS resolved_count,
       ROUND(AVG(CAST(resolved_date AS DATE) - CAST(created_date AS DATE)), 2) AS avg_days_to_resolve
  FROM complaint
 WHERE status = 'Resolved'
 GROUP BY category
HAVING AVG(CAST(resolved_date AS DATE) - CAST(created_date AS DATE)) > 2
 ORDER BY avg_days_to_resolve DESC;
```

category	resolved count	avg days to resolve
Carpentry	47	3.26
Other	32	2.31
Plumbing	67	2.04

Q6. Unresolved complaints older than 3 days with student, room and assigned staff (5-table join, outer join)

```
SELECT c.complaint_id, c.category, c.priority, c.status,
       TRUNC(SYSDATE - CAST(c.created_date AS DATE)) AS days_open,
       s.name AS student, r.room_number, st.name AS staff_name
  FROM complaint c
  JOIN student s              ON s.student_id = c.student_id
  JOIN room_allocation a      ON a.student_id = s.student_id AND a.status = 'Active'
  JOIN room r                 ON r.room_id = a.room_id
  LEFT JOIN staff st          ON st.staff_id = c.assigned_to
 WHERE c.status <> 'Resolved'
   AND c.created_date < SYSTIMESTAMP - INTERVAL '3' DAY
 ORDER BY days_open DESC
 FETCH FIRST 12 ROWS ONLY;
```

complaint id	category	priority	status	days open	student	room number	staff name
55	Network	Low	In Progress	23	Ayaan Sethi	AR-103	Ayaan Sharma
10	Plumbing	High	Assigned	20	Manav Verma	SH-506	Siddharth Agarwal

complaint id	category	priority	status	days open	student	room number	staff name
44	Network	Low	Assigned	20	Nandini Das	KA-305	Shaurya Bhatt
341	Carpentry	Medium	In Progress	19	Mihir Reddy	SH-302	Aarav Naidu
266	Plumbing	High	In Progress	19	Anjali Das	KA-204	Ayaan Menon
407	Carpentry	High	Assigned	19	Aadhya Chopra	GA-305	Manav Yadav
385	Carpentry	High	Assigned	18	Rishi Krishnan	SH-404	Aarav Naidu
36	Electrical	Low	In Progress	17	Pranav Mishra	NI-302	Deepak Nair
420	Plumbing	High	In Progress	17	Reyansh Subramanian	NI-501	Kabir Agarwal
52	Carpentry	Medium	In Progress	16	Akash Pandey	NI-203	Naveen Bhatt
14	Electrical	Medium	In Progress	16	Siddharth Chauhan	VI-203	Rajesh Mehta
70	Housekeeping	High	In Progress	16	Gaurav Mehta	NI-304	Dev Rao

(first 12 rows shown)

Q7. Maintenance staff workload (LEFT JOIN + GROUP BY + function)

```
SELECT st.staff_id, st.name AS staff_name, st.specialization, h.hostel_name,
       COUNT(c.complaint_id) AS total_handled,
       SUM(CASE WHEN c.status = 'Resolved' THEN 1 ELSE 0 END) AS resolved,
       fn_staff_open_workload(st.staff_id) AS open_now
  FROM staff st
  JOIN hostel h          ON h.hostel_id = st.hostel_id
  LEFT JOIN complaint c  ON c.assigned_to = st.staff_id
 WHERE st.role = 'Maintenance'
 GROUP BY st.staff_id, st.name, st.specialization, h.hostel_name
 ORDER BY open_now DESC, total_handled DESC
 FETCH FIRST 12 ROWS ONLY;
```

staff id	staff name	specialization	hostel name	total handled	resolved	open now
46	Ayaan Menon	Plumbing	Kaveri Block	14	10	4
49	Shaurya Bhatt	Network	Kaveri Block	17	14	3
14	Kabir Agarwal	Plumbing	Nilgiri Block	14	11	3
31	Kunal Krishnan	Carpentry	Vindhya Block	13	10	3
23	Aarav Naidu	Carpentry	Shivalik Block	12	9	3
47	Dev Pillai	Carpentry	Kaveri Block	8	5	3
13	Deepak Nair	Electrical	Nilgiri Block	24	22	2
45	Yash Menon	Electrical	Kaveri Block	12	10	2
15	Naveen Bhatt	Carpentry	Nilgiri Block	12	10	2
9	Ayaan Sharma	Network	Aravali Block	12	10	2
10	Atharv Naidu	Plumbing	Aravali Block	9	7	2
26	Siddharth Agarwal	Plumbing	Shivalik Block	5	3	2

(first 12 rows shown)

Q8. Pending leave requests waiting for a warden (join + filter + sort)

```
SELECT l.leave_id, s.registration_no, s.name, h.hostel_name,
       l.start_date, l.end_date, l.end_date - l.start_date + 1 AS days, l.destination
  FROM leave_request l
  JOIN student s          ON s.student_id = l.student_id
  JOIN room_allocation a  ON a.student_id = s.student_id AND a.status = 'Active'
  JOIN room r             ON r.room_id = a.room_id
  JOIN hostel h           ON h.hostel_id = r.hostel_id
 WHERE l.status = 'Pending'
 ORDER BY l.start_date
 FETCH FIRST 12 ROWS ONLY;
```

leave id	registration no	name	hostel name	start date	end date	days	destination
141	25BME1046	Aditya Das	Nilgiri Block	08-10-2026	14-10-2026	7	Pune
182	24BIT1069	Priya Agarwal	Kaveri Block	08-10-2026	11-10-2026	4	Mumbai
222	25BAI1048	Prakash Verma	Nilgiri Block	08-10-2026	11-10-2026	4	Madurai
224	25BYB1044	Kabir Malhotra	Vindhya Block	09-10-2026	14-10-2026	6	Bhopal
143	26BYB1011	Naveen Singh	Aravali Block	10-10-2026	13-10-2026	4	Chandigarh
117	26BME1041	Tanmay Chopra	Nilgiri Block	10-10-2026	15-10-2026	6	Chandigarh
133	26BEC1045	Keerthana Patel	Ganga Block	11-10-2026	15-10-2026	5	Kolkata
3	24BYB1001	Aarav Mehta	Aravali Block	12-10-2026	15-10-2026	4	New Delhi

leave id	registration no	name	hostel name	start date	end date	days	destination
202	23BME1019	Prakash Bose	Aravali Block	12-10-2026	18-10-2026	7	Patna
4	24BYB1001	Aarav Mehta	Aravali Block	13-10-2026	16-10-2026	4	Mumbai
83	24BIT1071	Ishita Mehta	Ganga Block	15-10-2026	19-10-2026	5	New Delhi

Q9. Leave approval rate of each warden (GROUP BY + HAVING)

```
SELECT st.name AS staff_name, h.hostel_name,
       COUNT(*) AS decided,
       SUM(CASE WHEN l.status = 'Approved' THEN 1 ELSE 0 END) AS approved,
       ROUND(100 * SUM(CASE WHEN l.status = 'Approved' THEN 1 ELSE 0 END) / COUNT(*), 1) AS approval_pct
  FROM leave_request l
  JOIN staff st  ON st.staff_id = l.approved_by
  JOIN hostel h  ON h.hostel_id = st.hostel_id
 GROUP BY st.name, h.hostel_name
HAVING COUNT(*) >= 10
 ORDER BY decided DESC;
```

staff name	hostel name	decided	approved	approval pct
Muskan Deshmukh	Ganga Block	29	23	79.3
Karan Kumar	Vindhya Block	27	24	88.9
Aniket Jain	Shivalik Block	26	24	92.3
Divya Gill	Kaveri Block	23	19	82.6
Sarthak Bose	Nilgiri Block	23	21	91.3
Suresh Iyer	Aravali Block	23	21	91.3
Gaurav Sethi	Shivalik Block	21	19	90.5
Abhinav Rao	Vindhya Block	21	19	90.5
Tanmay Jain	Nilgiri Block	19	16	84.2
Karan Mishra	Aravali Block	19	17	89.5
Sneha Banerjee	Kaveri Block	18	15	83.3
Saanvi Krishnan	Ganga Block	10	8	80

(first 12 rows shown)

Q10. Visitors expected in the next 7 days with approved passes

```
SELECT v.pass_id, v.visit_date, v.visitor_name, v.relation_to_student, s.name AS student, v.qr_code
  FROM visitor_pass v JOIN student s ON s.student_id = v.student_id
 WHERE v.status = 'Approved'
   AND v.visit_date BETWEEN TRUNC(SYSDATE) AND TRUNC(SYSDATE) + 7
 ORDER BY v.visit_date, v.visitor_name;
```

pass id	visit date	visitor name	relation to student	student	qr code
258	08-10-2026	Ananya Menon	Cousin	Aarav Reddy	SHMS-VP-258-5D459ED1
2	09-10-2026	Siddharth Reddy	Friend	Aarav Mehta	SHMS-VP-2-5D459ED1
39	10-10-2026	Gaurav Verma	Friend	Om Gupta	SHMS-VP-39-5D459ED1
22	10-10-2026	Manav Rathore	Uncle	Kavya Mehta	SHMS-VP-22-5D459ED1
34	11-10-2026	Suresh Jain	Uncle	Ishaan Sharma	SHMS-VP-34-5D459ED1
54	11-10-2026	Swati Reddy	Sister	Kunal Reddy	SHMS-VP-54-5D459ED1
21	11-10-2026	Vikram Bose	Brother	Vedant Bose	SHMS-VP-21-5D459ED1
46	13-10-2026	Aditi Menon	Mother	Aditi Menon	SHMS-VP-46-5D459ED1
181	13-10-2026	Rishi Verma	Guardian	Sarthak Jain	SHMS-VP-181-5D459ED1
91	13-10-2026	Saanvi Kulkarni	Guardian	Karan Agarwal	SHMS-VP-91-5D459ED1
132	14-10-2026	Riya Das	Aunt	Rajesh Mukherjee	SHMS-VP-132-5D459ED1
20	14-10-2026	Simran Mehta	Sister	Prakash Mehta	SHMS-VP-20-5D459ED1

(first 12 rows shown)

Q11. Visitor passes per month and status (date grouping)

```
SELECT TO_CHAR(visit_date, 'YYYY-MM') AS month,
       COUNT(*) AS requests,
       SUM(CASE WHEN status IN ('Approved', 'Completed') THEN 1 ELSE 0 END) AS approved,
       SUM(CASE WHEN status = 'Rejected' THEN 1 ELSE 0 END) AS rejected,
       SUM(CASE WHEN status = 'Pending' THEN 1 ELSE 0 END) AS pending
  FROM visitor_pass
 GROUP BY TO_CHAR(visit_date, 'YYYY-MM')
 ORDER BY month;
```

month	requests	approved	rejected	pending
2026-07	32	30	2	0
2026-08	81	70	11	0
2026-09	85	72	13	0
2026-10	72	43	2	27

Q12. Students waiting for a room (NOT EXISTS subquery)

```
SELECT s.registration_no, s.name, s.gender, s.year, s.branch
  FROM student s
 WHERE NOT EXISTS (SELECT 1 FROM room_allocation a
                   WHERE a.student_id = s.student_id AND a.status = 'Active')
 ORDER BY s.year, s.registration_no
 FETCH FIRST 12 ROWS ONLY;
```

registration no	name	gender	year	branch
26BAI1089	Arjun Patel	Male	1	CSE (Artificial Intelligence)
26BCE1060	Priya Menon	Female	1	Computer Science and Engineering
26BEC1034	Vikram Naidu	Male	1	Electronics and Communication
26BME1044	Arjun Rao	Male	1	Mechanical Engineering
26BME1055	Pranav Singh	Male	1	Mechanical Engineering
26BYB1025	Siddharth Mukherjee	Male	1	CSE (Cyber Security)
26BYB1030	Sahil Shah	Male	1	CSE (Cyber Security)
26BYB1060	Shaurya Shah	Male	1	CSE (Cyber Security)
25BAI1085	Swati Patel	Female	2	CSE (Artificial Intelligence)
25BCE1020	Tanmay Deshmukh	Male	2	Computer Science and Engineering
25BME1006	Om Das	Male	2	Mechanical Engineering
25BME1052	Aishwarya Tiwari	Female	2	Mechanical Engineering

(first 12 rows shown)

Q13. Students who raised more complaints than the average student (nested subquery in HAVING)

```
SELECT s.registration_no, s.name, COUNT(c.complaint_id) AS complaints
  FROM student s JOIN complaint c ON c.student_id = s.student_id
 GROUP BY s.registration_no, s.name
HAVING COUNT(c.complaint_id) > (SELECT AVG(cnt) FROM (SELECT COUNT(*) AS cnt FROM complaint GROUP BY
  student_id))
 ORDER BY complaints DESC
 FETCH FIRST 12 ROWS ONLY;
```

registration no	name	complaints
24BYB1001	Aarav Mehta	6
24BCE1005	Rahul Das	5
25BIT1078	Reyansh Yadav	5
26BME1049	Dev Deshmukh	5
25BEC1051	Vedant Banerjee	5
23BIT1003	Reyansh Subramanian	5
26BME1008	Abhinav Sethi	5
23BEC1010	Vedant Rathore	4
24BYB1025	Vaishnavi Rao	4
24BME1034	Vikram Saxena	4
24BCE1024	Rajesh Mukherjee	4
23BME1071	Ishita Mukherjee	4

(first 12 rows shown)

Q14. Rooms that are completely full in the hostel with the highest occupancy (subquery in WHERE)

```
SELECT h.hostel_name, r.room_number, r.room_type, r.occupied_count || '/' || r.capacity AS occupancy
  FROM room r JOIN hostel h ON h.hostel_id = r.hostel_id
 WHERE r.occupied_count = r.capacity
   AND r.hostel_id = (SELECT hostel_id FROM hostel
                      ORDER BY fn_hostel_occupancy_pct(hostel_id) DESC FETCH FIRST 1 ROW ONLY)
 ORDER BY r.room_number;
```

hostel name	room number	room type	occupancy
Nilgiri Block	NI-103	Double	2/2
Nilgiri Block	NI-104	Double	2/2
Nilgiri Block	NI-202	Single	1/1
Nilgiri Block	NI-203	Triple	3/3
Nilgiri Block	NI-204	Double	2/2
Nilgiri Block	NI-205	Double	2/2
Nilgiri Block	NI-206	Double	2/2
Nilgiri Block	NI-301	Triple	3/3
Nilgiri Block	NI-302	Double	2/2
Nilgiri Block	NI-303	Double	2/2
Nilgiri Block	NI-304	Triple	3/3
Nilgiri Block	NI-305	Triple	3/3

(first 12 rows shown)

Q15. Room-type utilisation across all hostels (GROUP BY + ratio)

```
SELECT room_type, COUNT(*) AS rooms, SUM(capacity) AS beds, SUM(occupied_count) AS occupied,
       ROUND(100 * SUM(occupied_count) / SUM(capacity), 1) AS utilisation_pct
  FROM room
 GROUP BY room_type
 ORDER BY MIN(capacity);
```

room type	rooms	beds	occupied	utilisation pct
Single	17	17	16	94.1
Double	65	130	114	87.7
Triple	71	213	150	70.4
Quad	27	108	55	50.9

Q16. Students with outstanding fees for the current semester (extension table, join + aggregate)

```
SELECT s.registration_no, s.name,
       SUM(f.amount_due) AS due, SUM(f.amount_paid) AS paid,
       SUM(f.amount_due - f.amount_paid) AS outstanding
  FROM fee_payment f JOIN student s ON s.student_id = f.student_id
 WHERE f.semester = '2026-27 Odd'
 GROUP BY s.registration_no, s.name
HAVING SUM(f.amount_due - f.amount_paid) > 0
 ORDER BY outstanding DESC
 FETCH FIRST 12 ROWS ONLY;
```

registration no	name	due	paid	outstanding
26BAI1111	Suresh Patel	101500	10000	91500
24BYB1009	Vedant Gupta	91500	0	91500
24BEC1021	Varun Mehta	109500	27500	82000
23BYB1065	Meera Kumar	72500	0	72500
25BME1060	Nandini Saxena	72500	0	72500
25BAI1043	Akash Joshi	91500	27500	64000
26BME1106	Nandini Singh	101500	37500	64000
24BYB1041	Sneha Sethi	91500	27500	64000
23BME1036	Nikhil Kapoor	91500	27500	64000
25BEC1095	Diya Krishnan	91500	27500	64000
23BME1059	Ishita Bose	91500	27500	64000
26BAI1100	Yash Subramanian	101500	37500	64000

(first 12 rows shown)

Q17. Hostel staff directory / emergency contacts (wardens and maintenance per hostel)

```
SELECT h.hostel_name, st.role, NVL(st.specialization, '-') AS specialization, st.name AS staff_name, st.phone
  FROM staff st JOIN hostel h ON h.hostel_id = st.hostel_id
 WHERE h.hostel_id = 1
 ORDER BY CASE st.role WHEN 'Warden' THEN 1 WHEN 'Admin' THEN 2 ELSE 3 END, st.specialization;
```

hostel name	role	specialization	staff name	phone
Aravali Block	Warden	-	Karan Mishra	8682052589
Aravali Block	Warden	-	Suresh Iyer	7646861642
Aravali Block	Admin	-	Meera Krishnan	9718841330
Aravali Block	Admin	-	Rajesh Kulkarni	9370283822
Aravali Block	Maintenance	Carpentry	Ishaan Bose	6621052185
Aravali Block	Maintenance	Electrical	Prakash Naidu	8247764725
Aravali Block	Maintenance	Housekeeping	Manav Subramanian	6452441064
Aravali Block	Maintenance	Network	Ayaan Sharma	6130411895
Aravali Block	Maintenance	Plumbing	Atharv Naidu	6659397615
Aravali Block	Maintenance	Plumbing	Naveen Chopra	6948939494

Q18. Branch-wise residents and average CGPA of current residents (IN subquery + GROUP BY)

```
SELECT branch, COUNT(*) AS residents, ROUND(AVG(cgpa), 2) AS avg_cgpa
  FROM student
 WHERE student_id IN (SELECT student_id FROM room_allocation WHERE status = 'Active')
 GROUP BY branch
 ORDER BY residents DESC;
```

branch	residents	avg cgpa
CSE (Artificial Intelligence)	73	7.86
Information Technology	57	7.89
CSE (Cyber Security)	55	7.94
Mechanical Engineering	55	7.83
Electronics and Communication	51	7.91
Computer Science and Engineering	44	7.84

Q19. INSERT a new student registration

```
INSERT INTO student (registration_no, name, email, phone, gender, year, branch, cgpa, password_hash)
VALUES ('26BYB1999', 'Demo New Student', 'demo.new@student.shms.local', '9000000099', 'Male', 1,
        'CSE (Cyber Security)', 8.10, '$2b$10$demoHashOnlyForThisInsertxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx');
SELECT student_id, registration_no, name, year FROM student WHERE registration_no = '26BYB1999';
```

Result: 1 row(s) affected

student id	registration no	name	year
361	26BYB1999	Demo New Student	1

Q20. UPDATE: warden reschedules all Assigned complaints of Aravali Block to tomorrow

```
UPDATE complaint c
   SET scheduled_date = TRUNC(SYSDATE) + 1
 WHERE c.status = 'Assigned'
   AND c.assigned_to IN (SELECT staff_id FROM staff WHERE hostel_id = 1);
```

Result: 4 row(s) affected

Q21. DELETE: remove withdrawn marketplace listings older than 30 days

```
DELETE FROM marketplace_listing
 WHERE status = 'Withdrawn'
   AND listed_date < SYSDATE - 30;
```

Result: 2 row(s) affected

9. Procedures

Four stored procedures implement the main business operations. They do not COMMIT: the caller (Spring's `@Transactional` service method, or the demo script) commits or rolls back, so an error raised anywhere undoes the whole operation.

Procedure	Purpose	Called by the application
sp_allocate_room	Validates student, staff role, room (row lock), gender and capacity; picks a free bed; inserts the allocation; returns allocation id and bed (OUT).	POST /api/allocations
sp_vacate_room	Closes an Active allocation (status Vacated, vacate date).	PUT /api/allocations/{id}/vacate
sp_assign_complaint	Assigns a complaint after checking role, specialization and hostel.	PUT /api/complaints/{id}/assign
sp_decide_leave	Pending → Approved/Rejected by a warden or admin only.	PUT /api/leaves/{id}/decision

sp_allocate_room

```
-- 1. Allocate a room to a student.
--    Student -> verify student -> verify room -> verify gender & capacity
--    -> pick free bed -> insert allocation (trigger updates occupancy)
CREATE OR REPLACE PROCEDURE sp_allocate_room (
    p_student_id     IN  NUMBER,
    p_room_id        IN  NUMBER,
    p_staff_id       IN  NUMBER,
    p_allocation_id  OUT NUMBER,
    p_bed_number     OUT NUMBER
)
IS
    v_student_gender  student.gender%TYPE;
    v_hostel_gender   hostel.gender_type%TYPE;
    v_staff_role      staff.role%TYPE;
    v_active_count    NUMBER;
    v_dummy           NUMBER;
BEGIN
    -- verify student
    BEGIN
        SELECT gender INTO v_student_gender FROM student WHERE student_id = p_student_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20010, 'Student ' || p_student_id || ' does not exist');
    END;

    SELECT COUNT(*) INTO v_active_count
      FROM room_allocation
     WHERE student_id = p_student_id AND status = 'Active';
    IF v_active_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20011, 'Student already has an active room allocation');
    END IF;

    -- verify staff member doing the allocation
    BEGIN
        SELECT role INTO v_staff_role FROM staff WHERE staff_id = p_staff_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20014, 'Staff ' || p_staff_id || ' does not exist');
    END;
    IF v_staff_role NOT IN ('Admin', 'Warden') THEN
        RAISE_APPLICATION_ERROR(-20014, 'Only an Admin or Warden can allocate rooms');
    END IF;

    -- verify room and lock its row, so two allocations cannot fill the same bed at once
    BEGIN
        SELECT r.room_id, h.gender_type
          INTO v_dummy, v_hostel_gender
          FROM room r JOIN hostel h ON h.hostel_id = r.hostel_id
         WHERE r.room_id = p_room_id
           FOR UPDATE OF r.occupied_count;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20012, 'Room ' || p_room_id || ' does not exist');
    END;

    IF v_hostel_gender <> v_student_gender THEN
        RAISE_APPLICATION_ERROR(-20013, 'Room is in a ' || v_hostel_gender || ' hostel; student is ' ||
  v_student_gender);
    END IF;

    -- verify capacity
    IF fn_available_beds(p_room_id) <= 0 THEN
        RAISE_APPLICATION_ERROR(-20015, 'Room is full');
    END IF;

    p_bed_number := fn_find_free_bed(p_room_id);

    INSERT INTO room_allocation (student_id, room_id, allocated_by, bed_number, allocation_date, status)
    VALUES (p_student_id, p_room_id, p_staff_id, p_bed_number, SYSDATE, 'Active')
    RETURNING allocation_id INTO p_allocation_id;
    -- trg_allocation_occupancy increases room.occupied_count here
END sp_allocate_room;
/
```

Execution output (T1)

```
===== T1. ROOM ALLOCATION (sp_allocate_room + trg_allocation_validate + trg_allocation_occupancy) =====
Student : 26BYB1025 Siddharth Mukherjee
Room    : AR-202 (Triple) occupied BEFORE = 1/3
Result    : allocation_id = 419, bed = 2
Room    : occupied AFTER  = 2/3  (updated by trigger)
Rule checks:
same student again    -> ORA-20011: Student already has an active room allocation
female -> male hostel -> ORA-20013: Room is in a Male hostel; student is Female
room already full     -> ORA-20015: Room is full
maintenance allocates -> ORA-20014: Only an Admin or Warden can allocate rooms
```

sp_vacate_room

```
-- 2. Vacate a room: mark the active allocation as Vacated.
--    trg_allocation_occupancy then frees the bed (occupied_count - 1).
CREATE OR REPLACE PROCEDURE sp_vacate_room (p_allocation_id IN NUMBER)
IS
    v_status room_allocation.status%TYPE;
BEGIN
    BEGIN
        SELECT status INTO v_status FROM room_allocation WHERE allocation_id = p_allocation_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20020, 'Allocation ' || p_allocation_id || ' does not exist');
    END;

    IF v_status <> 'Active' THEN
        RAISE_APPLICATION_ERROR(-20021, 'Allocation is already ' || v_status);
    END IF;

    UPDATE room_allocation
       SET status = 'Vacated',
           vacate_date = SYSDATE
     WHERE allocation_id = p_allocation_id;
END sp_vacate_room;
/
```

Execution output (T2)

```
===== T2. VACATE ROOM (sp_vacate_room + trg_allocation_occupancy) =====
Allocation 85 vacated. Room 55 occupied: 3 -> 2
Vacate again -> ORA-20021: Allocation is already Vacated
```

sp_assign_complaint

```
-- 3. Assign a complaint to a maintenance staff member.
--    Checks: complaint is still open/assigned, staff is Maintenance,
--    specialization matches the category ('Other' can go to anyone),
--    and staff works in the hostel where the student currently lives.
CREATE OR REPLACE PROCEDURE sp_assign_complaint (
    p_complaint_id    IN NUMBER,
    p_staff_id        IN NUMBER,
    p_scheduled_date  IN DATE DEFAULT NULL
)
IS
    v_status          complaint.status%TYPE;
    v_category        complaint.category%TYPE;
    v_student_id      complaint.student_id%TYPE;
    v_role            staff.role%TYPE;
    v_specialization  staff.specialization%TYPE;
    v_staff_hostel    staff.hostel_id%TYPE;
    v_student_hostel  hostel.hostel_id%TYPE;
BEGIN
    BEGIN
        SELECT status, category, student_id
          INTO v_status, v_category, v_student_id
          FROM complaint WHERE complaint_id = p_complaint_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20030, 'Complaint ' || p_complaint_id || ' does not exist');
    END;

    IF v_status NOT IN ('Open', 'Assigned') THEN
        RAISE_APPLICATION_ERROR(-20030, 'Complaint is already ' || v_status || ' and cannot be reassigned');
    END IF;

    BEGIN
        SELECT role, specialization, hostel_id
          INTO v_role, v_specialization, v_staff_hostel
          FROM staff WHERE staff_id = p_staff_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20031, 'Staff ' || p_staff_id || ' does not exist');
    END;

    IF v_role <> 'Maintenance' THEN
        RAISE_APPLICATION_ERROR(-20031, 'Complaints can only be assigned to maintenance staff');
    END IF;

    IF v_category <> 'Other' AND v_specialization <> v_category THEN
        RAISE_APPLICATION_ERROR(-20032, 'A ' || v_category || ' complaint cannot be assigned to a '
                                        || v_specialization || ' specialist');
    END IF;

    -- hostel check (only if the student currently has a room)
    BEGIN
        SELECT r.hostel_id INTO v_student_hostel
          FROM room_allocation a JOIN room r ON r.room_id = a.room_id
         WHERE a.student_id = v_student_id AND a.status = 'Active';

        IF v_student_hostel <> v_staff_hostel THEN
            RAISE_APPLICATION_ERROR(-20033, 'Staff member works in a different hostel');
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN NULL;
    END;

    UPDATE complaint
       SET assigned_to    = p_staff_id,
           status         = 'Assigned',
           scheduled_date = NVL(p_scheduled_date, scheduled_date)
     WHERE complaint_id = p_complaint_id;
END sp_assign_complaint;
/
```

sp_decide_leave

```
-- 4. Approve or reject a pending leave request.
CREATE OR REPLACE PROCEDURE sp_decide_leave (
    p_leave_id  IN NUMBER,
    p_staff_id  IN NUMBER,
    p_decision  IN VARCHAR2
)
IS
    v_status  leave_request.status%TYPE;
    v_role    staff.role%TYPE;
BEGIN
    IF p_decision NOT IN ('Approved', 'Rejected') THEN
        RAISE_APPLICATION_ERROR(-20040, 'Decision must be Approved or Rejected');
    END IF;

    BEGIN
        SELECT status INTO v_status FROM leave_request WHERE leave_id = p_leave_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20041, 'Leave request ' || p_leave_id || ' does not exist');
    END;

    IF v_status <> 'Pending' THEN
        RAISE_APPLICATION_ERROR(-20041, 'Leave request is already ' || v_status);
    END IF;

    BEGIN
        SELECT role INTO v_role FROM staff WHERE staff_id = p_staff_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20042, 'Staff ' || p_staff_id || ' does not exist');
    END;

    IF v_role NOT IN ('Warden', 'Admin') THEN
        RAISE_APPLICATION_ERROR(-20042, 'Only a Warden or Admin can decide leave requests');
    END IF;

    UPDATE leave_request
       SET status = p_decision,
           approved_by = p_staff_id
     WHERE leave_id = p_leave_id;
END sp_decide_leave;
/
```

Execution output (T4)

```
===== T4. LEAVE APPROVAL (sp_decide_leave) =====
Leave 3 (12-Oct to 15-Oct, New Delhi) status = Pending
Maintenance staff decides -> ORA-20042: Only a Warden or Admin can decide leave requests
After warden decision: status = Approved, approved_by = 2
Decide again -> ORA-20041: Leave request is already Approved
```

10. Functions

Function	Returns	Used for
fn_available_beds(room_id)	capacity − occupied	Vacancy lists, allocation check, available-rooms query (Q2)
fn_hostel_occupancy_pct(hostel_id)	% of beds occupied	Dashboard, hostel report, Q3
fn_staff_open_workload(staff_id)	open jobs of a worker	Choosing staff for a complaint, auto-assign cursor, Q7
fn_find_free_bed(room_id)	lowest free bed number	Inside sp_allocate_room

fn_available_beds and fn_find_free_bed

```
-- 1. Number of free beds in a room (capacity minus current occupancy).
--    Used by: vacancy screens, sp_allocate_room, available-room queries.
CREATE OR REPLACE FUNCTION fn_available_beds (p_room_id IN NUMBER)
RETURN NUMBER
IS
    v_capacity  room.capacity%TYPE;
    v_occupied  room.occupied_count%TYPE;
BEGIN
    SELECT capacity, occupied_count
      INTO v_capacity, v_occupied
      FROM room
     WHERE room_id = p_room_id;

    RETURN v_capacity - v_occupied;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20001, 'Room ' || p_room_id || ' does not exist');
END fn_available_beds;
/

-- 4. Lowest bed number in a room that is not taken by an active allocation.
--    Returns NULL when the room is full. Used by sp_allocate_room.
CREATE OR REPLACE FUNCTION fn_find_free_bed (p_room_id IN NUMBER)
RETURN NUMBER
IS
    v_capacity room.capacity%TYPE;
    v_taken    NUMBER;
BEGIN
    SELECT capacity INTO v_capacity FROM room WHERE room_id = p_room_id;

    FOR bed IN 1 .. v_capacity LOOP
        SELECT COUNT(*)
          INTO v_taken
          FROM room_allocation
         WHERE room_id = p_room_id
           AND bed_number = bed
           AND status = 'Active';

        IF v_taken = 0 THEN
            RETURN bed;
        END IF;
    END LOOP;

    RETURN NULL;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20001, 'Room ' || p_room_id || ' does not exist');
END fn_find_free_bed;
/
```

Output

room number	capacity	occupied count	available beds	next free bed
AR-101	2	2	0	
AR-102	3	3	0	
AR-103	3	3	0	
AR-104	4	4	0	
AR-105	2	2	0	
AR-106	2	2	0	

fn_hostel_occupancy_pct

```
-- 2. Occupancy percentage of a hostel (occupied beds / total beds * 100).
--    Used by: admin dashboard tiles and hostel reports.
CREATE OR REPLACE FUNCTION fn_hostel_occupancy_pct (p_hostel_id IN NUMBER)
RETURN NUMBER
IS
    v_total_beds     NUMBER;
    v_occupied_beds  NUMBER;
BEGIN
    SELECT NVL(SUM(capacity), 0), NVL(SUM(occupied_count), 0)
      INTO v_total_beds, v_occupied_beds
      FROM room
     WHERE hostel_id = p_hostel_id;

    IF v_total_beds = 0 THEN
        RETURN 0;
    END IF;

    RETURN ROUND(v_occupied_beds * 100 / v_total_beds, 2);
END fn_hostel_occupancy_pct;
/
```

Output

hostel name	occupancy pct
Aravali Block	67.86
Nilgiri Block	79.45
Shivalik Block	67.9
Vindhya Block	76.32
Ganga Block	65.48
Kaveri Block	74.29

fn_staff_open_workload

```
-- 3. Number of complaints a maintenance staff member is currently working on.
--    Used by: complaint assignment screen (pick the least busy person)
--             and the auto-assign cursor procedure.
CREATE OR REPLACE FUNCTION fn_staff_open_workload (p_staff_id IN NUMBER)
RETURN NUMBER
IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*)
      INTO v_count
      FROM complaint
     WHERE assigned_to = p_staff_id
       AND status IN ('Assigned', 'In Progress');

    RETURN v_count;
END fn_staff_open_workload;
/
```

Output (maintenance staff of Aravali Block)

staff id	name	specialization	open jobs
3	Prakash Naidu	Electrical	1
6	Naveen Chopra	Plumbing	1
7	Ishaan Bose	Carpentry	1
8	Manav Subramanian	Housekeeping	1
9	Ayaan Sharma	Network	2
10	Atharv Naidu	Plumbing	2

11. Cursor

sp_auto_assign_open_complaints uses an explicit cursor (OPEN – FETCH loop – CLOSE) to process every open, unassigned complaint, High priority and oldest first. For each row it finds the least busy maintenance worker with the right specialization in the student's hostel (ordered by `fn_staff_open_workload`) and calls `sp_assign_complaint`. It is the *Auto-assign open complaints* button for the warden.

```
-- 1. Auto-assign all open, unassigned complaints.
--    An explicit cursor walks through the open complaints (High priority
--    and oldest first). For each one it finds the least busy maintenance
--    staff member with the right specialization in the student's hostel
--    (using fn_staff_open_workload) and calls sp_assign_complaint.
--    Used by the admin "Auto-assign" button.
CREATE OR REPLACE PROCEDURE sp_auto_assign_open_complaints (
    p_assigned_count  OUT NUMBER,
    p_skipped_count   OUT NUMBER
)
IS
    CURSOR c_open_complaints IS
        SELECT c.complaint_id, c.category,
               (SELECT r.hostel_id
                  FROM room_allocation a JOIN room r ON r.room_id = a.room_id
                 WHERE a.student_id = c.student_id AND a.status = 'Active') AS hostel_id
          FROM complaint c
         WHERE c.status = 'Open'
           AND c.assigned_to IS NULL
         ORDER BY CASE c.priority WHEN 'High' THEN 1 WHEN 'Medium' THEN 2 ELSE 3 END,
                  c.created_date;

    v_complaint  c_open_complaints%ROWTYPE;
    v_staff_id   staff.staff_id%TYPE;
BEGIN
    p_assigned_count := 0;
    p_skipped_count  := 0;

    OPEN c_open_complaints;
    LOOP
        FETCH c_open_complaints INTO v_complaint;
        EXIT WHEN c_open_complaints%NOTFOUND;

        BEGIN
            SELECT staff_id INTO v_staff_id
              FROM (SELECT s.staff_id
                      FROM staff s
                     WHERE s.role = 'Maintenance'
                       AND (v_complaint.category = 'Other' OR s.specialization = v_complaint.category)
                       AND (v_complaint.hostel_id IS NULL OR s.hostel_id = v_complaint.hostel_id)
                     ORDER BY fn_staff_open_workload(s.staff_id), s.staff_id)
             WHERE ROWNUM = 1;

            sp_assign_complaint(v_complaint.complaint_id, v_staff_id);
            p_assigned_count := p_assigned_count + 1;
            DBMS_OUTPUT.PUT_LINE('Complaint ' || v_complaint.complaint_id || ' (' || v_complaint.category
                                 || ') -> staff ' || v_staff_id);
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                p_skipped_count := p_skipped_count + 1;
                DBMS_OUTPUT.PUT_LINE('Complaint ' || v_complaint.complaint_id
                                     || ' skipped: no matching staff available');
        END;
    END LOOP;
    CLOSE c_open_complaints;
END sp_auto_assign_open_complaints;
/
```

Execution output (T6)

```
===== T6. AUTO-ASSIGN OPEN COMPLAINTS (cursor sp_auto_assign_open_complaints) =====
Complaint 248 (Carpentry) -> staff 39
Complaint 6 (Plumbing) -> staff 6
Complaint 164 (Plumbing) -> staff 22
Complaint 279 (Plumbing) -> staff 30
Complaint 171 (Electrical) -> staff 3
Complaint 69 (Network) -> staff 41
Complaint 277 (Electrical) -> staff 29
Complaint 421 (Plumbing) -> staff 14
Complaint 118 (Other) -> staff 24
Complaint 5 (Electrical) -> staff 3
Complaint 132 (Network) -> staff 49
Complaint 152 (Carpentry) -> staff 39
Complaint 442 (Housekeeping) -> staff 18
Complaint 456 (Carpentry) -> staff 31
Complaint 338 (Housekeeping) -> staff 24
Complaint 238 (Plumbing) -> staff 22
Complaint 415 (Electrical) -> staff 21
Complaint 134 (Carpentry) -> staff 23
Complaint 68 (Housekeeping) -> staff 8
Open complaints: 19 -> 0   (assigned 19, skipped 0)
```

sp_hostel_report uses a cursor FOR loop over hostels and a **parameterised cursor** over the room types of each hostel to print an occupancy report. The application reads its DBMS_OUTPUT lines and shows them on the Reports page.

```
-- 2. Hostel occupancy and complaint report (parameterised cursor inside a cursor FOR loop).
--    Prints one block per hostel with room-type breakdown, occupancy % and pending complaints.
CREATE OR REPLACE PROCEDURE sp_hostel_report
IS
    CURSOR c_hostels IS
        SELECT hostel_id, hostel_name, gender_type FROM hostel ORDER BY hostel_id;

    CURSOR c_room_types (p_hostel_id NUMBER) IS
        SELECT room_type, COUNT(*) AS rooms, SUM(capacity) AS beds, SUM(occupied_count) AS occupied
          FROM room
         WHERE hostel_id = p_hostel_id
         GROUP BY room_type
         ORDER BY MIN(capacity);

    v_pending NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('===== SMART HOSTEL OCCUPANCY REPORT (' || TO_CHAR(SYSDATE, 'DD-Mon-YYYY') || ')
  =====');

    FOR h IN c_hostels LOOP
        SELECT COUNT(*) INTO v_pending
          FROM complaint c
          JOIN room_allocation a ON a.student_id = c.student_id AND a.status = 'Active'
          JOIN room r ON r.room_id = a.room_id
         WHERE r.hostel_id = h.hostel_id
           AND c.status <> 'Resolved';

        DBMS_OUTPUT.PUT_LINE(CHR(10) || h.hostel_name || ' (' || h.gender_type || ')  occupancy: '
                             || fn_hostel_occupancy_pct(h.hostel_id) || '%   pending complaints: ' ||
  v_pending);

        FOR t IN c_room_types(h.hostel_id) LOOP
            DBMS_OUTPUT.PUT_LINE('   ' || RPAD(t.room_type, 8) || ' rooms: ' || LPAD(t.rooms, 3)
                                 || '   beds: ' || LPAD(t.beds, 4) || '   occupied: ' || LPAD(t.occupied, 4)
                                 || '   free: ' || LPAD(t.beds - t.occupied, 4));
        END LOOP;
    END LOOP;
END sp_hostel_report;
/
```

Execution output (T7)

```
===== T7. HOSTEL REPORT (cursor sp_hostel_report + fn_hostel_occupancy_pct) =====
===== SMART HOSTEL OCCUPANCY REPORT (08-Oct-2026) =====
Aravali Block (Male)  occupancy: 69.05%   pending complaints: 12
Single     rooms:   1   beds:    1   occupied:    1   free:    0
Double     rooms:   8   beds:   16   occupied:   15   free:    1
Triple     rooms:  17   beds:   51   occupied:   33   free:   18
Quad     rooms:   4   beds:   16   occupied:    9   free:    7
Nilgiri Block (Male)  occupancy: 78.08%   pending complaints: 11
Single     rooms:   3   beds:    3   occupied:    3   free:    0
Double     rooms:  14   beds:   28   occupied:   25   free:    3
Triple     rooms:  10   beds:   30   occupied:   22   free:    8
Quad     rooms:   3   beds:   12   occupied:    7   free:    5
Shivalik Block (Male)  occupancy: 67.9%   pending complaints: 12
Single     rooms:   1   beds:    1   occupied:    1   free:    0
Double     rooms:  13   beds:   26   occupied:   20   free:    6
Triple     rooms:  10   beds:   30   occupied:   19   free:   11
Quad     rooms:   6   beds:   24   occupied:   15   free:    9
Vindhya Block (Male)  occupancy: 76.32%   pending complaints: 9
Single     rooms:   5   beds:    5   occupied:    5   free:    0
Double     rooms:   9   beds:   18   occupied:   18   free:    0
Triple     rooms:  11   beds:   33   occupied:   25   free:    8
Quad     rooms:   5   beds:   20   occupied:   10   free:   10
Ganga Block (Female)  occupancy: 65.48%   pending complaints: 6
Single     rooms:   2   beds:    2   occupied:    1   free:    1
Double     rooms:   8   beds:   16   occupied:   13   free:    3
Triple     rooms:  14   beds:   42   occupied:   31   free:   11
Quad     rooms:   6   beds:   24   occupied:   10   free:   14
Kaveri Block (Female)  occupancy: 74.29%   pending complaints: 14
Single     rooms:   5   beds:    5   occupied:    5   free:    0
Double     rooms:  13   beds:   26   occupied:   23   free:    3
Triple     rooms:   9   beds:   27   occupied:   20   free:    7
Quad     rooms:   3   beds:   12   occupied:    4   free:    8
```

12. Triggers

Trigger	Timing / event	Automatic action
trg_allocation_validate	BEFORE INSERT on ROOM_ALLOCATION	Rejects a bed number larger than the room capacity.
trg_allocation_occupancy	AFTER INSERT / DELETE / UPDATE OF status on ROOM_ALLOCATION	Adds or subtracts 1 from ROOM.occupied_count; the CHECK on ROOM stops overflow.
trg_complaint_status	BEFORE INSERT / UPDATE on COMPLAINT	Open → Assigned when staff is set; blocks In Progress/Resolved without staff; stamps resolved_date.
trg_visitor_pass_qr	BEFORE INSERT / UPDATE OF status on VISITOR_PASS	Generates a unique QR code value when a pass is approved.

trg_allocation_occupancy

```
-- 2. Keep ROOM.occupied_count in step with active allocations.
--    New active allocation -> +1, allocation leaves 'Active' -> -1.
--    If a room would go over capacity, the CHECK constraint ck_room_occupied
--    on ROOM rejects the update and the whole statement is rolled back.
CREATE OR REPLACE TRIGGER trg_allocation_occupancy
AFTER INSERT OR DELETE OR UPDATE OF status ON room_allocation
FOR EACH ROW
BEGIN
    IF INSERTING AND :NEW.status = 'Active' THEN
        UPDATE room SET occupied_count = occupied_count + 1 WHERE room_id = :NEW.room_id;

    ELSIF UPDATING AND :OLD.status = 'Active' AND :NEW.status <> 'Active' THEN
        UPDATE room SET occupied_count = occupied_count - 1 WHERE room_id = :OLD.room_id;

    ELSIF UPDATING AND :OLD.status <> 'Active' AND :NEW.status = 'Active' THEN
        UPDATE room SET occupied_count = occupied_count + 1 WHERE room_id = :NEW.room_id;

    ELSIF DELETING AND :OLD.status = 'Active' THEN
        UPDATE room SET occupied_count = occupied_count - 1 WHERE room_id = :OLD.room_id;
    END IF;
END;
/
```

trg_allocation_validate

```
-- 1. Validate a new allocation before it is saved:
--    the bed number must exist in the room (bed_number <= capacity).
CREATE OR REPLACE TRIGGER trg_allocation_validate
BEFORE INSERT ON room_allocation
FOR EACH ROW
DECLARE
    v_capacity room.capacity%TYPE;
BEGIN
    SELECT capacity INTO v_capacity FROM room WHERE room_id = :NEW.room_id;

    IF :NEW.bed_number > v_capacity THEN
        RAISE_APPLICATION_ERROR(-20016, 'Bed ' || :NEW.bed_number || ' does not exist in a room of capacity '
  || v_capacity);
    END IF;
END;
/
```

trg_complaint_status

```
-- 3. Complaint status rules:
--    * giving an Open complaint a staff member makes it 'Assigned'
--    * a complaint cannot be In Progress / Resolved without a staff member
--    * moving to 'Resolved' stamps resolved_date automatically
--    * moving away from 'Resolved' (re-opening) clears resolved_date
CREATE OR REPLACE TRIGGER trg_complaint_status
BEFORE INSERT OR UPDATE ON complaint
FOR EACH ROW
BEGIN
    IF :NEW.assigned_to IS NOT NULL AND :NEW.status = 'Open' THEN
        :NEW.status := 'Assigned';
    END IF;

    IF :NEW.status IN ('In Progress', 'Resolved') AND :NEW.assigned_to IS NULL THEN
        RAISE_APPLICATION_ERROR(-20050, 'Complaint must be assigned before it can be ' || :NEW.status);
    END IF;

    IF :NEW.status = 'Resolved' AND :NEW.resolved_date IS NULL THEN
        :NEW.resolved_date := SYSTIMESTAMP;
    ELSIF :NEW.status <> 'Resolved' THEN
        :NEW.resolved_date := NULL;
    END IF;
END;
/
```

Execution output (T3: complaint lifecycle)

```
===== T3. COMPLAINT LIFECYCLE (INSERT -> sp_assign_complaint -> UPDATE, trg_complaint_status) =====
Raised        : complaint 461 status = Open (DEFAULT)
Start work before assignment -> ORA-20050: Complaint must be assigned before it can be In Progress
Assign to plumber -> ORA-20032: A Electrical complaint cannot be assigned to a Plumbing specialist
Assigned    : staff 3, status = Assigned, scheduled 09-Oct-2026, staff workload now 2
Resolved    : status = Resolved, resolved_date set by trigger = YES
```

trg_visitor_pass_qr

```
-- 4. Generate a unique QR code value when a visitor pass is approved.
--    The frontend draws the QR image from this value; the gate verifies it.
CREATE OR REPLACE TRIGGER trg_visitor_pass_qr
BEFORE INSERT OR UPDATE OF status ON visitor_pass
FOR EACH ROW
BEGIN
    IF :NEW.status IN ('Approved', 'Completed') AND :NEW.qr_code IS NULL THEN
        :NEW.qr_code := 'SHMS-VP-' || :NEW.pass_id || '-' || SUBSTR(RAWTOHEX(SYS_GUID()), 1, 8);
    ELSIF :NEW.status IN ('Pending', 'Rejected') THEN
        :NEW.qr_code := NULL;
    END IF;
END;
/
```

Execution output (T5)

```
===== T5. VISITOR PASS APPROVAL (UPDATE + trg_visitor_pass_qr) =====
Requested : pass 271 status = Pending, qr_code = NULL
Approved  : qr_code = SHMS-VP-271-5D45A1A5  (generated by trigger)
Completed : entry 00:23, exit 02:23, qr_code kept = SHMS-VP-271-5D45A1A5
```

Status of all PL/SQL objects

object name	object type	status
FN_AVAILABLE_BEDS	FUNCTION	VALID
FN_FIND_FREE_BED	FUNCTION	VALID
FN_HOSTEL_OCCUPANCY_PCT	FUNCTION	VALID
FN_STAFF_OPEN_WORKLOAD	FUNCTION	VALID
SP_ALLOCATE_ROOM	PROCEDURE	VALID
SP_ASSIGN_COMPLAINT	PROCEDURE	VALID
SP_AUTO_ASSIGN_OPEN_COMPLAINTS	PROCEDURE	VALID
SP_DECIDE_LEAVE	PROCEDURE	VALID
SP_HOSTEL_REPORT	PROCEDURE	VALID
SP_VACATE_ROOM	PROCEDURE	VALID
TRG_ALLOCATION_OCCUPANCY	TRIGGER	VALID
TRG_ALLOCATION_VALIDATE	TRIGGER	VALID
TRG_COMPLAINT_STATUS	TRIGGER	VALID
TRG_VISITOR_PASS_QR	TRIGGER	VALID

13. CRUD Demonstrations

INSERT, UPDATE and DELETE are used throughout the application (registration, requests, approvals, notices, marketplace, inventory). The statements below were executed in SQL and rolled back.

Q19. INSERT a new student registration

```
INSERT INTO student (registration_no, name, email, phone, gender, year, branch, cgpa, password_hash)
VALUES ('26BYB1999', 'Demo New Student', 'demo.new@student.shms.local', '9000000099', 'Male', 1,
        'CSE (Cyber Security)', 8.10, '$2b$10$demoHashOnlyForThisInsertxxxxxxxxxxxxxxxxxxxxxxxxxxxxx');
SELECT student_id, registration_no, name, year FROM student WHERE registration_no = '26BYB1999';
```

Result: 1 row(s) affected

student id	registration no	name	year
361	26BYB1999	Demo New Student	1

Q20. UPDATE: warden reschedules all Assigned complaints of Aravali Block to tomorrow

```
UPDATE complaint c
   SET scheduled_date = TRUNC(SYSDATE) + 1
 WHERE c.status = 'Assigned'
   AND c.assigned_to IN (SELECT staff_id FROM staff WHERE hostel_id = 1);
```

Result: 4 row(s) affected

Q21. DELETE: remove withdrawn marketplace listings older than 30 days

```
DELETE FROM marketplace_listing
 WHERE status = 'Withdrawn'
   AND listed_date < SYSDATE - 30;
```

Result: 2 row(s) affected

Full create–update–delete cycle on an extension table (T9)

```
===== T9. INSERT / UPDATE / DELETE on an extension table =====
INSERT listing 43
UPDATE price -> 1 row
DELETE          -> 1 row
Rows left with that id: 0
```

CRUD through the application

Operation	Screen / endpoint	SQL executed by Hibernate
Create	Student raises a complaint — POST /api/complaints	INSERT INTO complaint …
Read	Complaints page — GET /api/complaints?status=Open	SELECT … FROM complaint JOIN student …
Update	Maintenance marks Resolved — PUT /api/complaints/{id}/status	UPDATE complaint SET status = 'Resolved' … (trigger stamps resolved_date)
Delete	Student withdraws a pending leave — DELETE /api/leaves/{id}	DELETE FROM leave_request WHERE leave_id = ?

14. Database Connectivity

The application connects to Oracle from Java (Spring Boot) through the Oracle JDBC driver (`ojdbc11`) and a HikariCP connection pool. The connection settings are in `application.yml` and can be overridden by environment variables; no credentials are written in Java code.

```
# ---------- Oracle (primary database) ----------
datasource:
  url: ${DB_URL:jdbc:oracle:thin:@localhost:1521/FREEPDB1}
  username: ${DB_USERNAME:user1}
  password: ${DB_PASSWORD:user1Password}
  hikari:
    maximum-pool-size: 10
```

Two access paths share the same connection and transaction:

- **Spring Data JPA** (Hibernate) for ordinary reads and writes of the 14 entity classes; `ddl-auto: validate` checks at start-up that every entity matches its Oracle table.
- **JdbcTemplate** in `PlsqlRepository` to call the procedures and functions, and in `ReportRepository` for report SQL.

Calling a stored procedure from Java

```
/** sp_allocate_room(student, room, staff, OUT allocation_id, OUT bed_number) */
public AllocationResult allocateRoom(long studentId, long roomId, long staffId) {
    return jdbcTemplate.execute((ConnectionCallback<AllocationResult>) con -> {
        try (CallableStatement cs = con.prepareCall("{call sp_allocate_room(?, ?, ?, ?, ?)}")) {
            cs.setLong(1, studentId);
            cs.setLong(2, roomId);
            cs.setLong(3, staffId);
            cs.registerOutParameter(4, Types.NUMERIC);
            cs.registerOutParameter(5, Types.NUMERIC);
            cs.execute();
            return new AllocationResult(cs.getLong(4), cs.getInt(5));
        }
    });
}
```

Flow of one interactive operation: React UI → Spring Boot → Oracle (SQL / PL/SQL) → result → Spring Boot → React UI. A rule violation inside PL/SQL (ORA-20xxx) is converted by `GlobalExceptionHandler` into HTTP 400 with the Oracle message, which the UI shows to the user.

Verification: the API test script `tools/api_workflow_test.py` performed 33 checks through the REST API — 33 passed, 0 failed. Its room-allocation part:

```
== 2. Room allocation (sp_allocate_room + occupancy trigger) ==
   waiting students: 25; using 26BAI1089 Arjun Patel
   room AR-202 before: occupied 1/3
[PASS] allocate room: HTTP 201 (expected 201) -> allocation 419, bed 2
   room AR-202 after : occupied 2/3 (updated by trigger)
[PASS] allocate same student again: HTTP 400 (expected 400) -> Student already has an active room allocation
[PASS] allocate female into male hostel: HTTP 400 (expected 400) -> Room is in a Male hostel; student is Female
[PASS] vacate room: HTTP 200 (expected 200) -> status Vacated, vacated 2026-10-07
   room AR-202 after vacate: occupied 1/3
[PASS] re-allocate (history row kept): HTTP 201 (expected 201) -> allocation 420
```

Running containers and connection health

Output captured from the running system: both containers healthy with restart policy `unless-stopped`, the persistent Oracle volume, and the backend's health endpoint reporting a working connection to Oracle (`db`) and Redis.

```
$ docker compose ps
NAME          IMAGE                         STATUS                    PORTS
shms-oracle   gvenzl/oracle-free:23-slim    Up 41 minutes (healthy)   0.0.0.0:1521->1521/tcp, [::]:1521->1521/tcp
shms-redis    redis:7.4-alpine              Up 59 minutes (healthy)   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp

$ docker volume ls --filter name=shms
DRIVER    VOLUME NAME
```

```
local     shms-oracle-data

$ docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' shms-oracle shms-redis
/shms-oracle unless-stopped
/shms-redis unless-stopped

$ curl http://localhost:8080/actuator/health
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP",
      "details": {
        "database": "Oracle",
        "validationQuery": "isValid()"
      }
    },
    "ping": {
      "status": "UP"
    },
    "redis": {
      "status": "UP",
      "details": {
        "version": "7.4.11"
      }
    }
  }
}
```

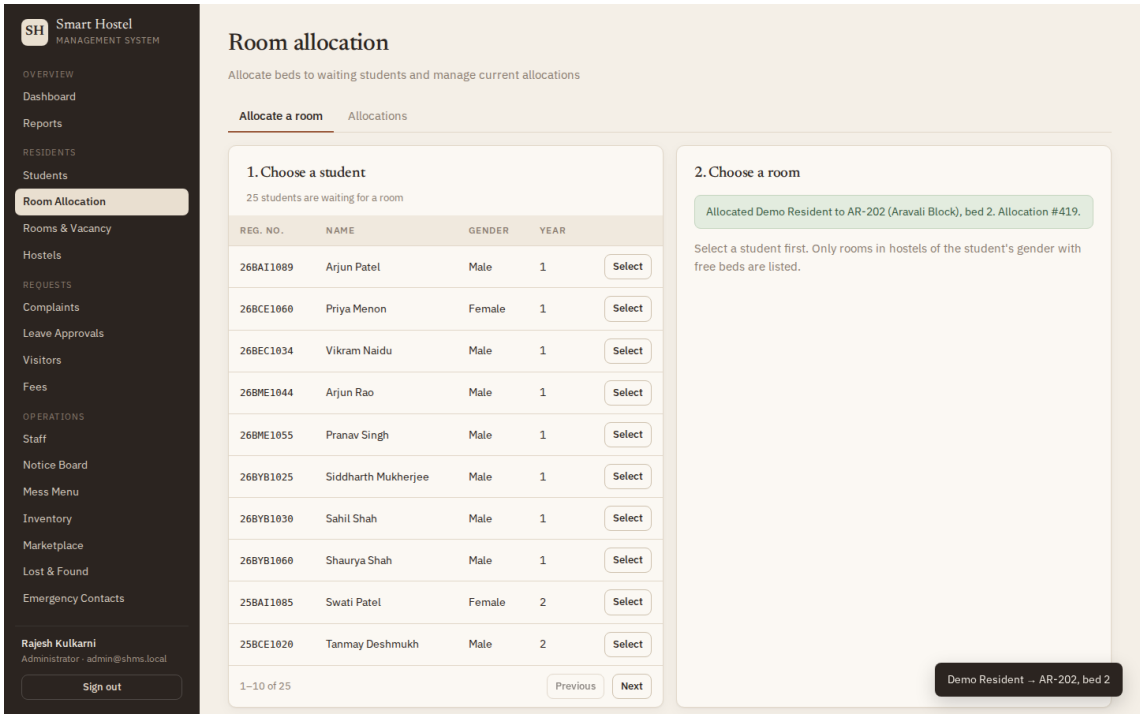


Figure 3: Allocation performed from the React UI; the confirmation shows the bed chosen by sp_allocate_room

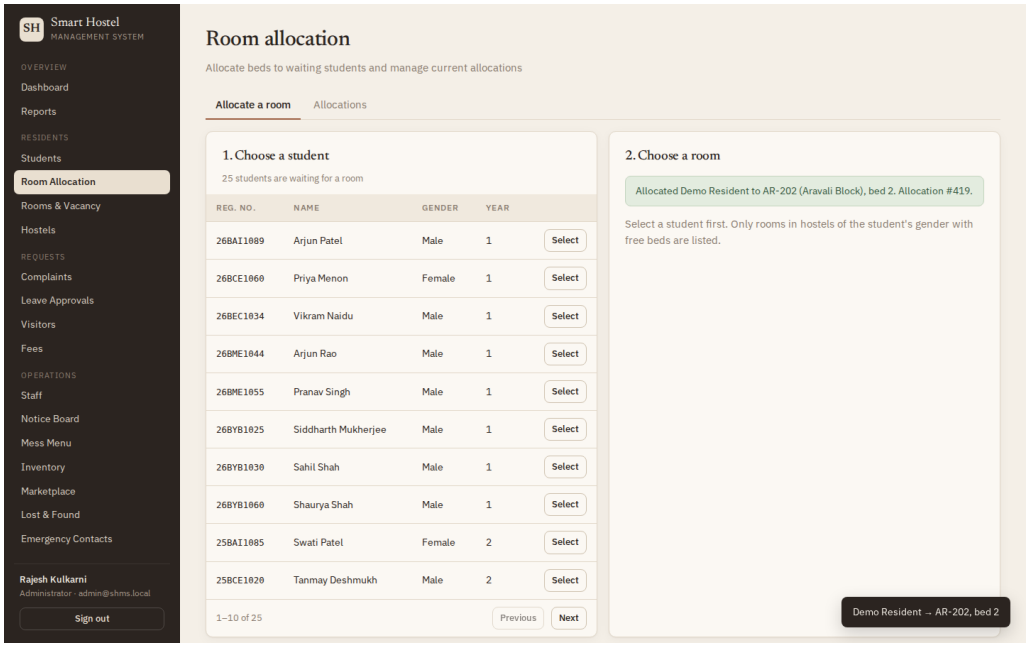
15. Real-World Transaction Demonstrations

For each major transaction the table states the operation, the tables involved, the SQL/PL-SQL component executed, how the database is affected and the result, followed by the actual output of `10_demo_transactions.sql` and, where useful, the screen from the application where the same transaction was performed (automated browser test `tools/ui_transactions_test.py`).

Transaction 1: Room allocation

Operation	Admin allocates a waiting student to a room.
Tables	STUDENT, STAFF, ROOM, HOSTEL, ROOM_ALLOCATION
SQL / PL-SQL	sp_allocate_room, fn_available_beds, fn_find_free_bed, trg_allocation_validate, trg_allocation_occupancy
Database effect	One ROOM_ALLOCATION row inserted (Active); ROOM.occupied_count +1; room row locked during the check.
Result	Allocation id and bed number returned; invalid requests rejected with ORA-20011/20013/20014/20015.

```
===== T1. ROOM ALLOCATION (sp_allocate_room + trg_allocation_validate + trg_allocation_occupancy) =====
Student : 26BYB1025 Siddharth Mukherjee
Room    : AR-202 (Triple) occupied BEFORE = 1/3
Result    : allocation_id = 419, bed = 2
Room    : occupied AFTER  = 2/3  (updated by trigger)
Rule checks:
same student again    -> ORA-20011: Student already has an active room allocation
female -> male hostel -> ORA-20013: Room is in a Male hostel; student is Female
room already full     -> ORA-20015: Room is full
maintenance allocates -> ORA-20014: Only an Admin or Warden can allocate rooms
```



Transaction 1: Room allocation page: confirmation of the allocation made by sp_allocate_room

Transaction 2: Vacating a room

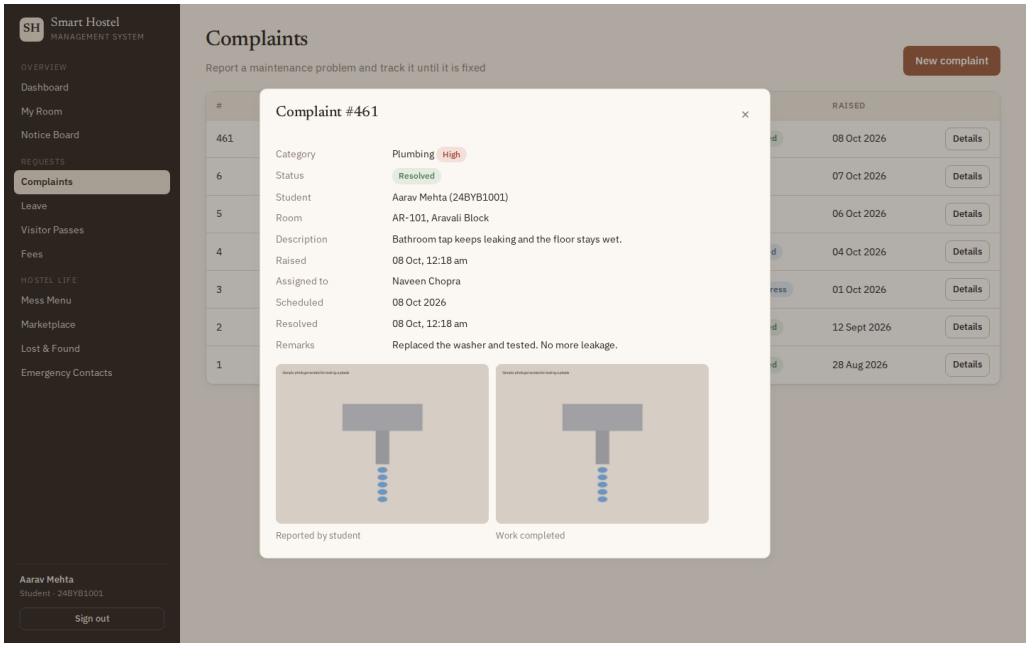
Operation	Admin vacates a student's room.
Tables	ROOM_ALLOCATION, ROOM
SQL / PL-SQL	sp_vacate_room, trg_allocation_occupancy
Database effect	Allocation status Active → Vacated, vacate_date set; occupied_count −1; history row kept.
Result	Bed becomes free; second vacate rejected (ORA-20021).

```
===== T2. VACATE ROOM (sp_vacate_room + trg_allocation_occupancy) =====
Allocation 85 vacated. Room 55 occupied: 3 -> 2
Vacate again -> ORA-20021: Allocation is already Vacated
```

Transaction 3: Complaint lifecycle

Operation	Student raises a complaint; warden assigns it; maintenance resolves it.
Tables	COMPLAINT, STAFF, ROOM_ALLOCATION, ROOM
SQL / PL-SQL	INSERT, sp_assign_complaint, fn_staff_open_workload, trg_complaint_status
Database effect	Complaint inserted (Open by DEFAULT) → assigned_to set (Assigned) → In Progress → Resolved, resolved_date stamped by trigger.
Result	Wrong specialist rejected (ORA-20032); starting work before assignment rejected (ORA-20050).

```
===== T3. COMPLAINT LIFECYCLE (INSERT -> sp_assign_complaint -> UPDATE, trg_complaint_status) =====
Raised        : complaint 461 status = Open (DEFAULT)
Start work before assignment -> ORA-20050: Complaint must be assigned before it can be In Progress
Assign to plumber -> ORA-20032: A Electrical complaint cannot be assigned to a Plumbing specialist
Assigned    : staff 3, status = Assigned, scheduled 09-Oct-2026, staff workload now 2
Resolved    : status = Resolved, resolved_date set by trigger = YES
```

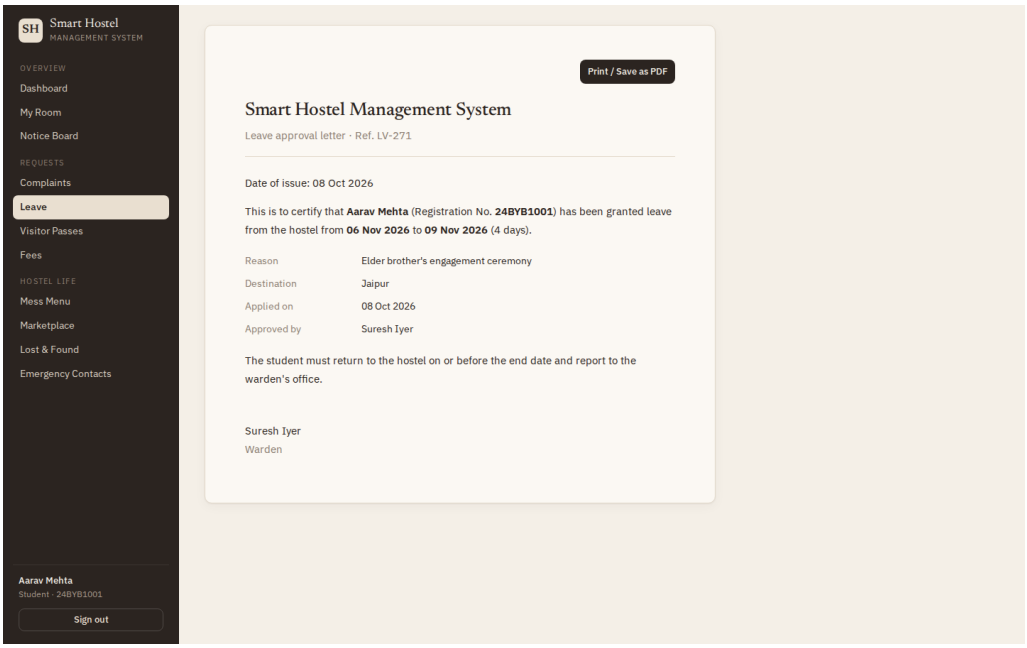


Transaction 3: Student's view of the resolved complaint; the resolved time was set by trg_complaint_status

Transaction 4: Leave approval

Operation	Student applies for leave; warden approves.
Tables	LEAVE_REQUEST, STAFF
SQL / PL-SQL	INSERT, sp_decide_leave
Database effect	Status Pending → Approved, approved_by set.
Result	Maintenance user rejected (ORA-20042); second decision rejected (ORA-20041); approval letter generated.

```
===== T4. LEAVE APPROVAL (sp_decide_leave) =====
Leave 3 (12-Oct to 15-Oct, New Delhi) status = Pending
Maintenance staff decides -> ORA-20042: Only a Warden or Admin can decide leave requests
After warden decision: status = Approved, approved_by = 2
Decide again -> ORA-20041: Leave request is already Approved
```

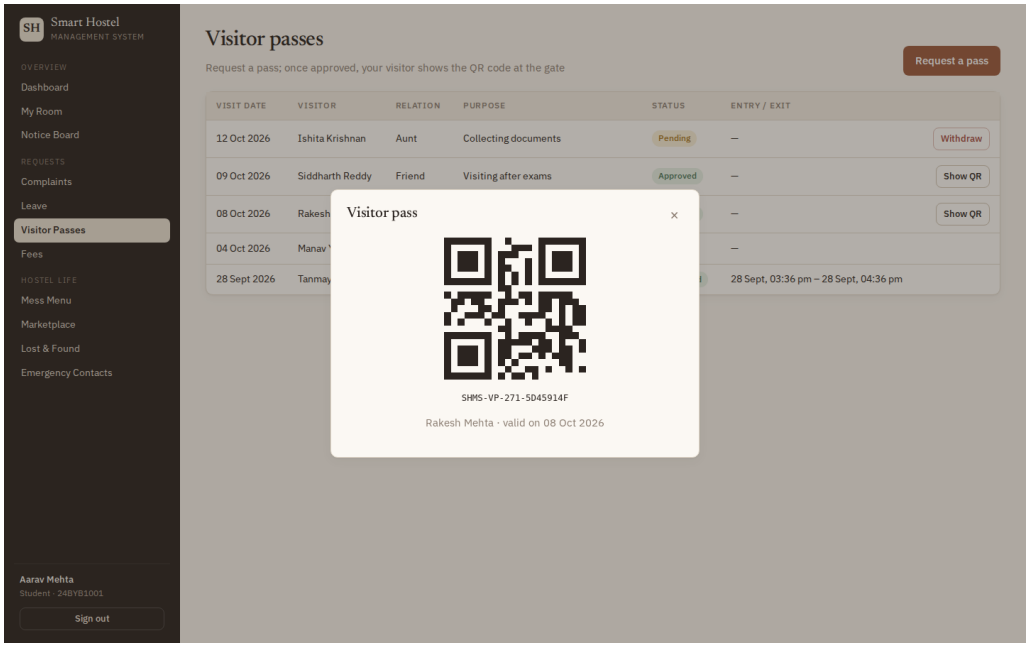


Transaction 4: Approval letter shown to the student after the warden's decision

Transaction 5: Visitor pass and gate entry

Operation	Student requests a pass; warden approves; visitor enters and leaves.
Tables	VISITOR_PASS
SQL / PL-SQL	INSERT, UPDATE, trg_visitor_pass_qr
Database effect	qr_code generated on approval; entry_time and exit_time recorded; status Completed.
Result	QR code shown to the student; reusing a completed pass rejected.

```
===== T5. VISITOR PASS APPROVAL (UPDATE + trg_visitor_pass_qr) =====
Requested : pass 271 status = Pending, qr_code = NULL
Approved  : qr_code = SHMS-VP-271-5D45A1A5  (generated by trigger)
Completed : entry 00:23, exit 02:23, qr_code kept = SHMS-VP-271-5D45A1A5
```

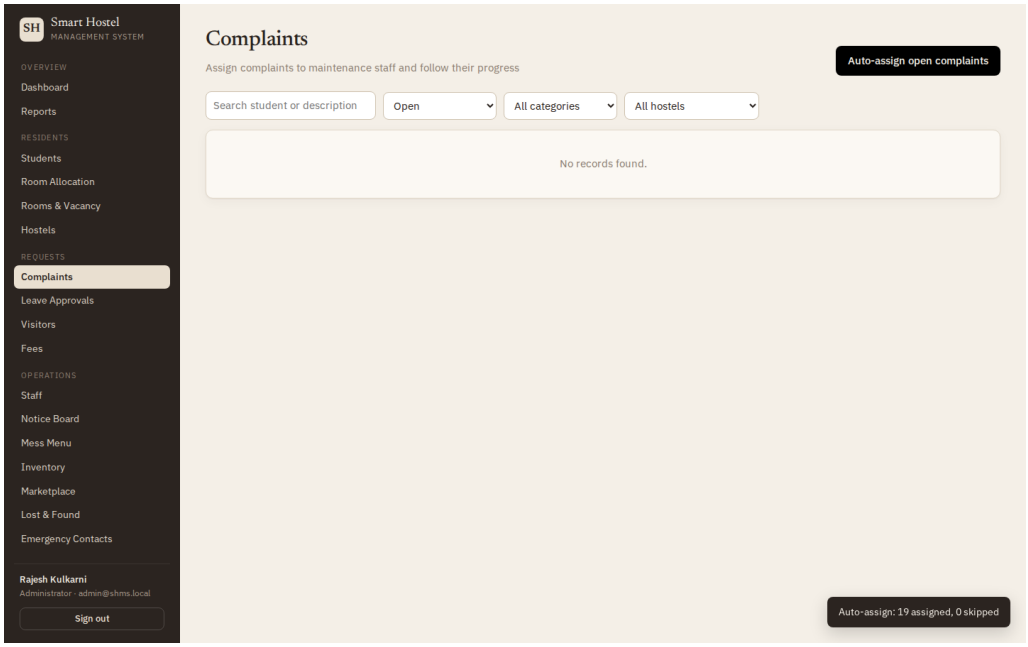


Transaction 5: QR code generated by trg_visitor_pass_qr, shown to the student

Transaction 6: Automatic complaint assignment

Operation	Warden assigns all open complaints at once.
Tables	COMPLAINT, STAFF, ROOM_ALLOCATION, ROOM
SQL / PL-SQL	Cursor procedure sp_auto_assign_open_complaints (calls sp_assign_complaint and fn_staff_open_workload)
Database effect	Every open complaint gets the least busy matching worker.
Result	Open complaints reduced to 0.

```
===== T6. AUTO-ASSIGN OPEN COMPLAINTS (cursor sp_auto_assign_open_complaints) =====
Complaint 248 (Carpentry) -> staff 39
Complaint 6 (Plumbing) -> staff 6
Complaint 164 (Plumbing) -> staff 22
Complaint 279 (Plumbing) -> staff 30
Complaint 171 (Electrical) -> staff 3
Complaint 69 (Network) -> staff 41
Complaint 277 (Electrical) -> staff 29
Complaint 421 (Plumbing) -> staff 14
Complaint 118 (Other) -> staff 24
Complaint 5 (Electrical) -> staff 3
Complaint 132 (Network) -> staff 49
Complaint 152 (Carpentry) -> staff 39
Complaint 442 (Housekeeping) -> staff 18
Complaint 456 (Carpentry) -> staff 31
Complaint 338 (Housekeeping) -> staff 24
Complaint 238 (Plumbing) -> staff 22
Complaint 415 (Electrical) -> staff 21
Complaint 134 (Carpentry) -> staff 23
Complaint 68 (Housekeeping) -> staff 8
Open complaints: 19 -> 0   (assigned 19, skipped 0)
```



Transaction 6: Complaints filtered by status Open after auto-assign: none remain; the message confirms 19 assigned

16. Problems Encountered and Solutions

These problems actually occurred during development. They are copied from the project's development log (`docs/development-log.md`), where each entry records the date, the error, the cause and the fix.

1. Seed script stopped on a duplicate registration number

Problem	Loading `08_seed_data.sql` stopped at the 122nd student.
Error	`ORA-00001: unique constraint (USER1.UQ_STUDENT_REGNO) violated on table USER1.STUDENT columns (REGISTRATION_NO)` `ORA-03301: (ORA-00001 details) row with column values (REGISTRATION_NO:'24BYB1001') already exists`
Cause	The seed generator gives the first student a fixed, documented registration number (`24BYB1001`, the demo student account). The random generator later produced the same number for another 3rd-year Cyber Security student. The `UNIQUE` constraint on `REGISTRATION_NO` correctly rejected it.
Solution	The generator now keeps a set of used registration numbers, with the demo number reserved from the start, and keeps drawing until it gets an unused one.

2. SQL*Plus committed half of a failed script

Problem	After the failure above, the database already contained 6 hostels and 121 students, although the script had stopped with an error.
Cause	`run-sql.sh` used `WHENEVER SQLERROR EXIT SQL.SQLCODE`. In SQL*Plus, `EXIT` **commits by default**, so every row inserted before the error was kept.
Solution	Changed the directive to `WHENEVER SQLERROR EXIT SQL.SQLCODE ROLLBACK`, so a failing script leaves no partial data. Added `drop-all.sql` + `reset-db.sh` to rebuild the schema cleanly.

3. Procedures, functions and triggers became INVALID after seeding

Problem	After `run-all.sh`, the verification query showed only 2/4 functions, 0/6 procedures and 1/4 triggers as `VALID`.
Cause	Seed rows are inserted with explicit IDs, so the end of the seed file runs `ALTER TABLE ... MODIFY (... GENERATED BY DEFAULT AS IDENTITY (START WITH LIMIT VALUE))` to move each identity sequence past those IDs. `ALTER TABLE` is DDL, and Oracle marks every PL/SQL object that depends on an altered table as `INVALID`. (Oracle would recompile them on first use, but they should not be left invalid.)
Solution	The seed file now ends with `EXEC DBMS_UTILITY.COMPILE_SCHEMA(schema => USER, compile_all => FALSE);`. After rebuilding: 4/4 functions, 6/6 procedures, 4/4 triggers `VALID`.

4. Synthetic data was unrealistic in ways the queries exposed

Problem	1. **Q5** (categories with average resolution time > 2 days, `HAVING`) returned *all six* categories with averages between 2.2 and 2.9 days, because every complaint got the same random 3–120 hour resolution time. 2. **Q6** (overdue complaints) listed the demo student's complaint as `Open` for 74 days, because the demo student's statuses were forced without adjusting the complaint dates. 3. **Q6** also showed a complaint "handled by" *Suresh Iyer*, the name of the demo **warden**. The demo staff names were hard-coded but not added to the generator's used-name set, so a random maintenance worker received the same name.
Solution	1. Resolution time now depends on the category (housekeeping 2–30 h … carpentry 24–168 h) and is halved for High priority. Q5 now returns only Carpentry (3.3 days) and Other (2.1 days). 2. The demo student's complaints are dated 40, 25, 6, 3, 1 and 0 days ago, matching their statuses. 3. All hard-coded demo names are reserved in the used-name set. Verified with a query: no duplicate names across STAFF and STUDENT, and 0 complaints assigned to non-maintenance staff. 4. Complaints older than 30 days are always Resolved.

5. Two demo checks tested the wrong rule

Problem	In `10_demo_transactions.sql`: - the "maintenance staff tries to allocate a room" check returned `ORA-20011 Student already has an active room allocation` instead of the staff-role error, because it reused the student who had just been allocated; `sp_allocate_room` checks the student before the staff member. - the "delete a hostel that still has rooms" check was blocked by `FK_MENU_HOSTEL` (an extension table) instead of a core foreign key, because Oracle reports whichever child table it finds first.
Solution	The role check now uses a different waiting student (result: `ORA-20014 Only an Admin or Warden can allocate rooms`). The FK check now deletes a room that has allocations (result: `ORA-02292 ... FK_ALLOC_ROOM`).

6. Resolving a complaint failed with a CHECK constraint (time-zone mismatch)

Problem	In `tools/api_workflow_test.py`, the maintenance user could start work on a complaint, but marking it **Resolved** returned HTTP 409.
Error	`SQL Error: 2290, SQLState: 23000` `ORA-02290: check constraint (USER1.CK_COMPLAINT_RESOLVED) violated`
Cause	The Oracle container ran in **UTC**, while the Spring Boot app runs in **IST (UTC+05:30)**. The app stored `created_date = 23:49` (IST). When the status became Resolved, the trigger `trg_complaint_status` set `resolved_date := SYSTIMESTAMP`, which the database evaluated as `18:19` (UTC). So `resolved_date < created_date`, and the CHECK constraint `resolved_date >= created_date` correctly rejected the update. Checked with `SELECT SYSTIMESTAMP FROM dual` (showed `+00:00`) vs. the host clock (IST).
Solution	Set `TZ: Asia/Kolkata` for the Oracle service in `database/docker-compose.yml` and recreated the container (data kept in the volume). `SYSTIMESTAMP` now returns `+05:30`, identical to the backend clock. This also prevents `SYSDATE`-based dates (allocation / vacate dates) from being a day behind between 00:00 and 05:30 IST.

7. Small bug in the Redis test script output

Problem	The first line of `tools/redis-demo-test.sh` printed `0` and then a stray `n/a` line.
Cause	`grep -c` prints `0` **and** exits with status 1 when nothing matches, so the `|| echo "n/a"` fallback ran as well.
Solution	Replaced `cmd || echo n/a` with an explicit `if [ -n "$LOG" ]; then ...; else ...; fi`.

8. Chart colour failed the colour-validation check

Problem	The UI accent colour `#9a5a3c` was going to be used for dashboard bar charts. The palette validator reported `[FAIL] Chroma floor — below floor (reads gray): #9a5a3c 0.095`.
Solution	Added a separate `--chart` colour `#a8532c`, which passed lightness, chroma and 3:1 contrast against the card surface `#fbf8f3`. Buttons keep the softer accent.

9. Stale search results in the React tables (race condition)

Problem	In the automated UI test, a visitor pass that existed in Oracle (`pass_id 271`, Pending) did not appear on the warden's Visitors page after searching for the visitor's name.
Cause	The `useApi` hook stored the response of every request. When the first (unfiltered) request answered *after* the second (filtered) one, it overwrote the table with old data. Typing quickly in any search box could cause the same problem.
Solution	`useApi` now marks each request as outdated when the URL changes (effect clean-up) and ignores answers from outdated requests. `reload()` re-runs the effect through a version counter.

10. Problems in the automated UI test itself

Problem	- The test looked for the newly registered student before the waiting-list table had loaded, so it paged past the student and failed on a disabled "Next" button → it now waits for the first table row. - The test leave reason ("Going home for Diwali") and visitor purpose also exist in the seed data, so the selector matched three rows (`strict mode violation`) → changed to texts that do not appear in the seed data.

11. Misleading Spring Data Redis start-up messages

Problem	At start-up Spring printed messages such as `Spring Data Redis - Could not safely identify store assignment for repository candidate interface ...VisitorPassRepository` for every JPA repository.
Cause	With both JPA and Redis on the classpath, Spring checks whether each repository might belong to Redis. This project uses Redis only through Spring Cache and `StringRedisTemplate`, never as a repository store.
Solution	`spring.data.redis.repositories.enabled: false` in `application.yml`. Start-up log now has no warnings.

12. Uploaded test images were committed to git (caught before pushing)

Problem	Before the first push, `git ls-files` showed 8 files under `uploads/complaints/` and `uploads/completed/` (photos uploaded by the automated UI test).
Cause	`app.upload-dir` is the relative path `uploads`, resolved from the backend's working directory. The jar had been started from the repository root, so images were saved to `./uploads/`, while `.gitignore` only covered `backend/uploads/` (the location when run from IntelliJ).
Solution	`.gitignore` now ignores `uploads/` at any level; the files were removed from the index (`git rm -r --cached uploads`) and the unpushed commit was amended. Nothing was pushed.

17. Conclusion

The database designed in Review 1 has been implemented in Oracle Database 23ai. The eight core tables were created with named PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL and DEFAULT constraints and populated with a realistic synthetic dataset (360 students, 180 rooms, 418 allocations, 460 complaints, 270 leave requests and 270 visitor passes).

21 SQL queries answer real questions of the hostel administration. Four procedures, four functions, two cursor procedures and four triggers implement the business operations and keep the data consistent automatically; all of them are called by the application. Six real-world transactions were demonstrated, including the rejection of every invalid request.

The database is connected to a Spring Boot application with a React user interface, so every operation shown here can also be performed from the web application. Review 3 will demonstrate the integrated system end to end.